

C++54期提高

Day01

一、面向对象设计基本概念

面向对象的分析（OOA）：解析出需求，要做什么。

面向对象的设计（OOD）：需要设计那些类，类中有哪些数据成员、哪些成员函数，类与类之间的关系。

面向对象的编程（OOP）：将需求与设计转换为代码。

UML语言：统一建模语言。

☒ 类图(class diagram)

- ☐ 对象图(object diagram)
- ☐ 用例图(use case diagram)
- ☐ 组件图(component diagram)
- ☐ 部署图(deployment diagram)
- ☐ 组合结构图(composite structure diagram)

☒ 序列图(sequence diagram)

- ☐ 协作图(collaboration diagram)
- ☐ 状态图(state diagram)
- ☐ 活动图(activity diagram)

静态

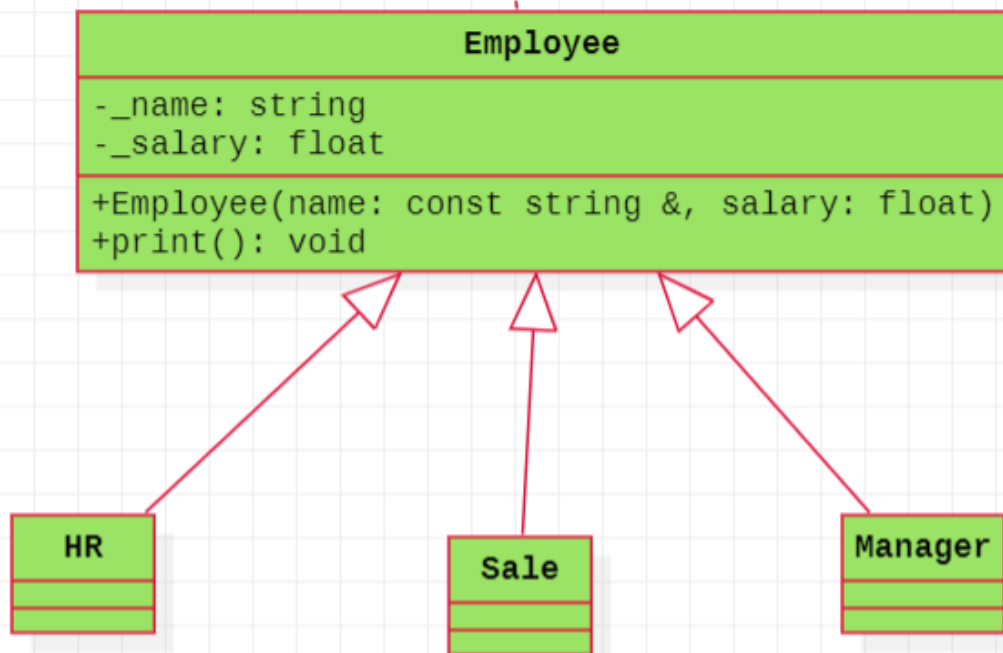
动态

二、类与类之间的关系（重要）

1、继承（泛化）

基类部分会成为派生类的一部分。在语义层面：A is B。在类图的画法上：从派生类向基类拉一个空心三角箭头。

类的三层结构，最上层是类的名字，
中间是类的数据成员，
最下层是成员函数



继承与泛化的区别

继承：先有基类，然后才有派生类。泛化：先有派生类，然后将具有相同属性的抽象成基类。

2、关联

2.1、双向的关联关系

彼此知道对方的存在，但是并不负责对方的生命周期。在语义层面：A has B。在代码层面上，可以使用指针或者引用。在类图的画法：就是一根实心直线。



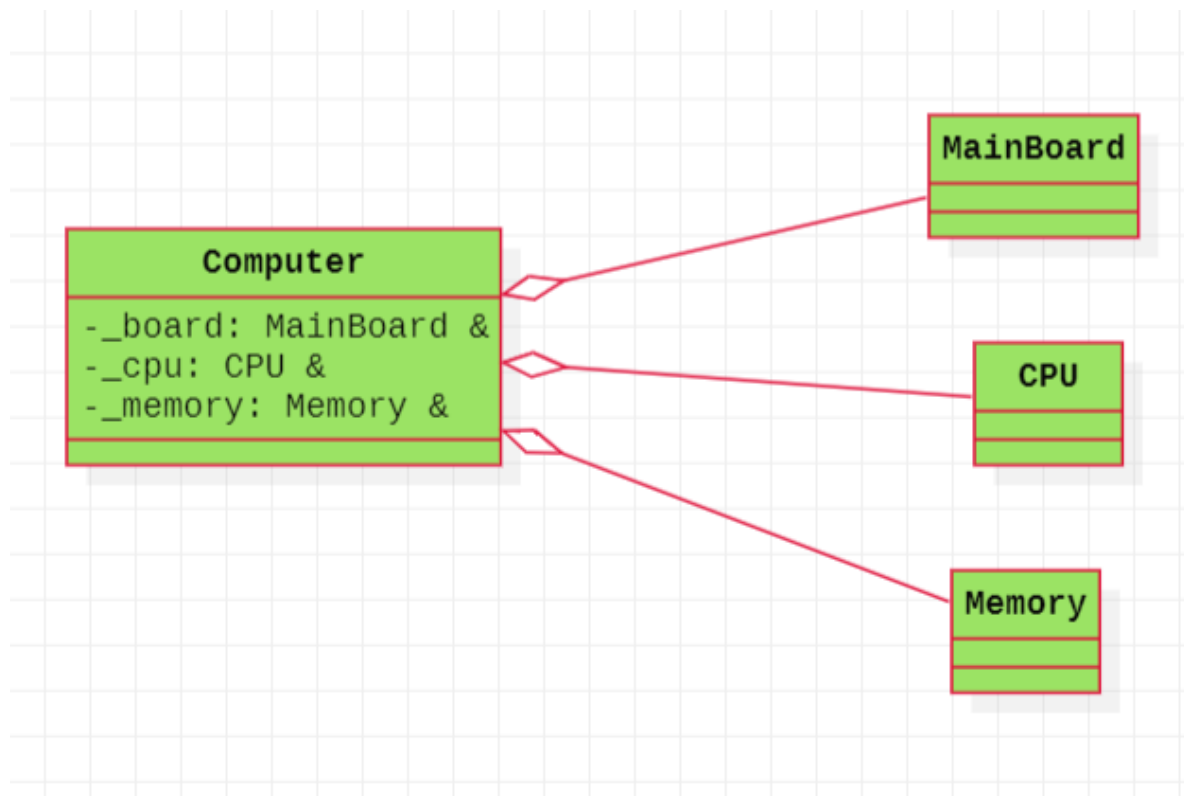
2.1、单向的关联关系

A知道B的存在，但是并不负责对方的生命周期。在语义层面：A has B。在代码层面上，可以使用指针或者引用。在类图的画法：从A指向B的实线箭头。



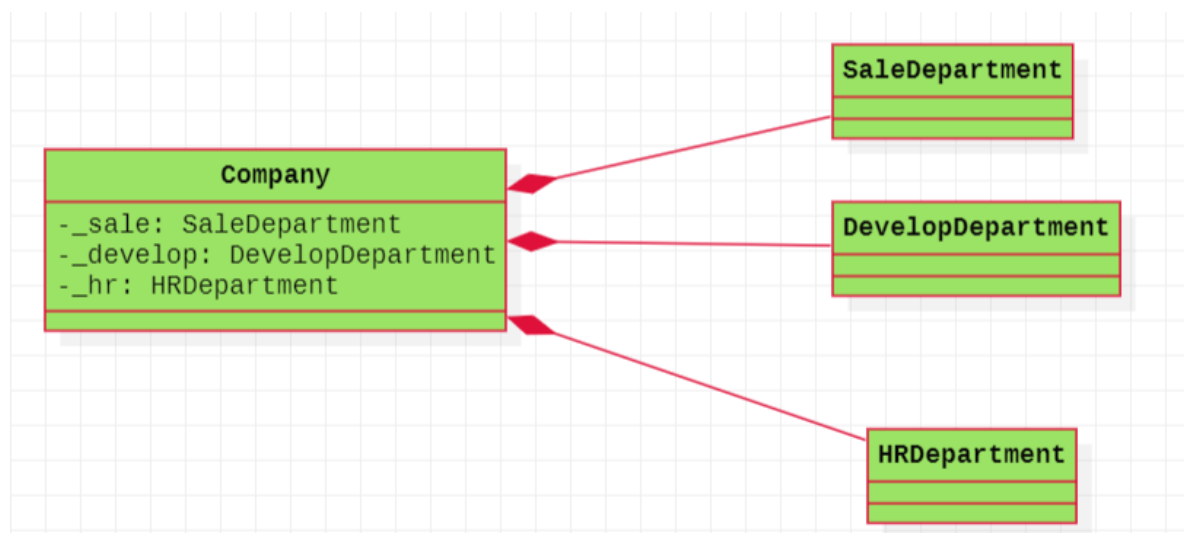
3、聚合

一种强一点的关联关系，有整体与局部的关系，但是整体并不负责局部的生命周期。在语义层面：A has B。在代码层面：使用的是指针或者引用。在类图的画法：可以从局部指向整体的空心菱形箭头。



4、组合

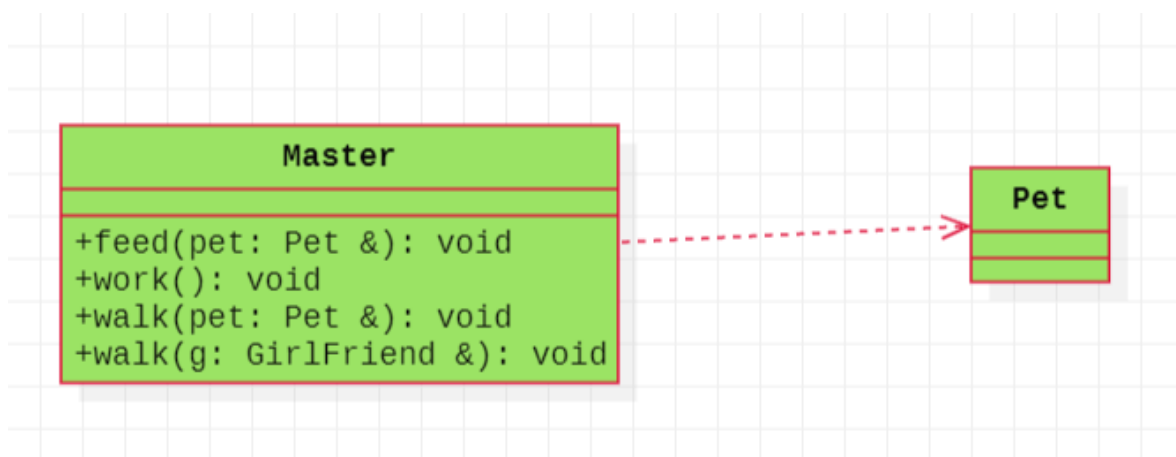
是一种更强的关联关系。有整体与局部的关系，但是整体负责局部的生命周期。在语义层面：A has B。在代码层面：使用的子对象。在类图的画法：可以从局部指向整体的实心菱形箭头。



5、依赖

是两个类之间的一种不确定的关系，语义上是一种A use B的关系，这种关系是**偶然的**，**临时的**，并非**固定**的。在类图的画法：使用虚线的箭头，从A指向B。在代码上表现为：

- B作为A的成员函数参数；
- B作为A的成员函数的局部变量（B作为A的成员函数的返回值）；
- A的成员函数调用B的静态方法。



总结:

- 1、从方向来看，继承是垂直关系，其它四种是横向关系。
- 2、从耦合程度来看，依赖 < 关联 < 聚合 < 组合 < 继承
- 3、从语义来看，继承 is，依赖 use，关联、聚合、组合 has
- 4、依赖关系主要在成员函数。其他的四种主要在数据成员的角度。

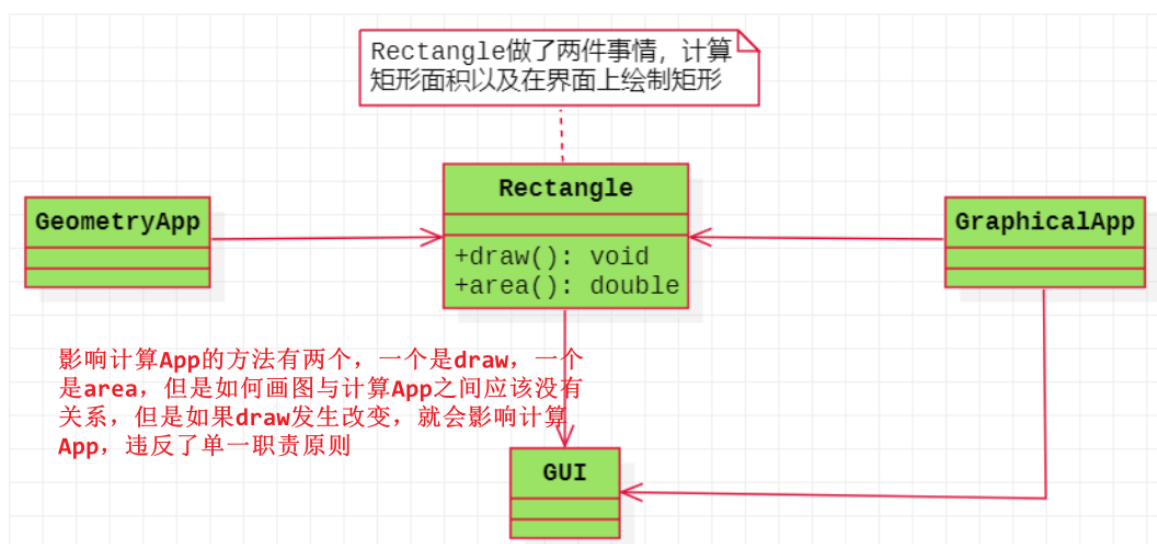
三、面向对象的设计原则（重要）

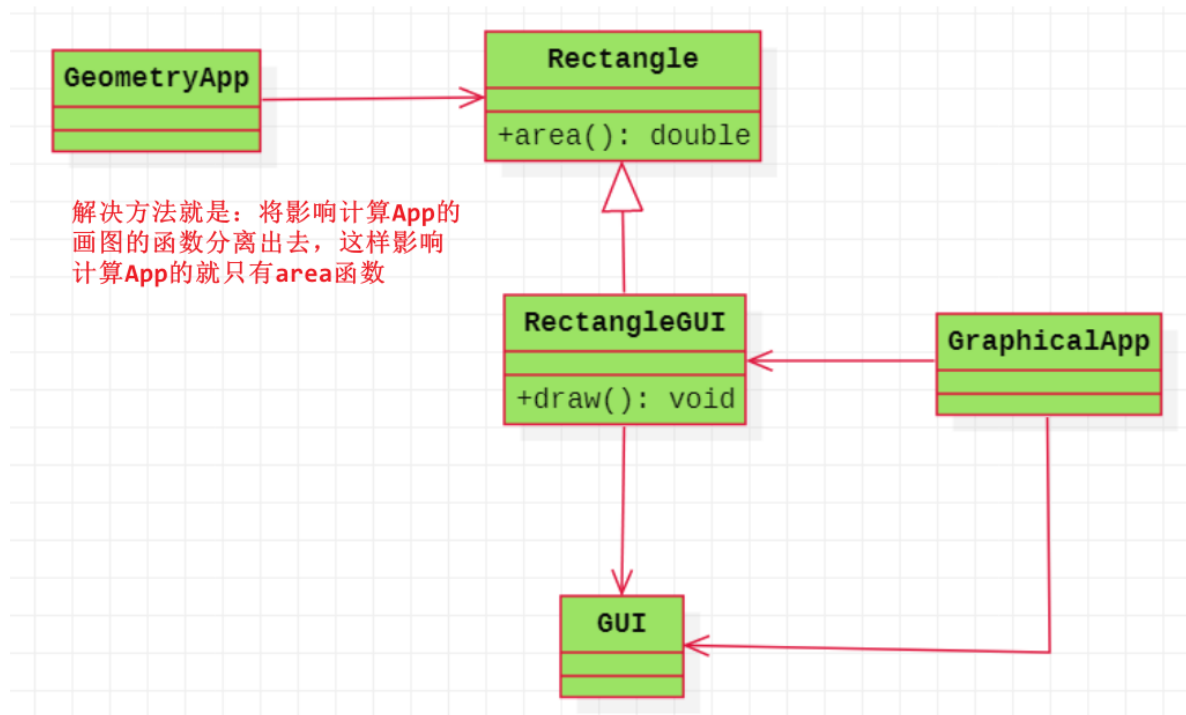
设计的目标：低耦合、高内聚。

耦合：强调是类与类之间、模块与模块之间的关系。内聚：强调的是类的内部或者模块内部的关系。

1、单一职责原则

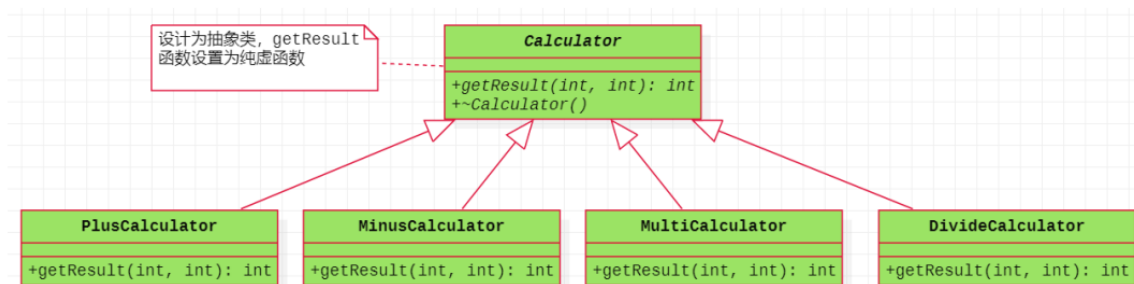
核心思想：一个类，最好只做一件事，只有一个引起它变化的原因





2、开闭原则（重要）

核心思想就是对抽象编程，而不对具体编程，因为抽象相对稳定

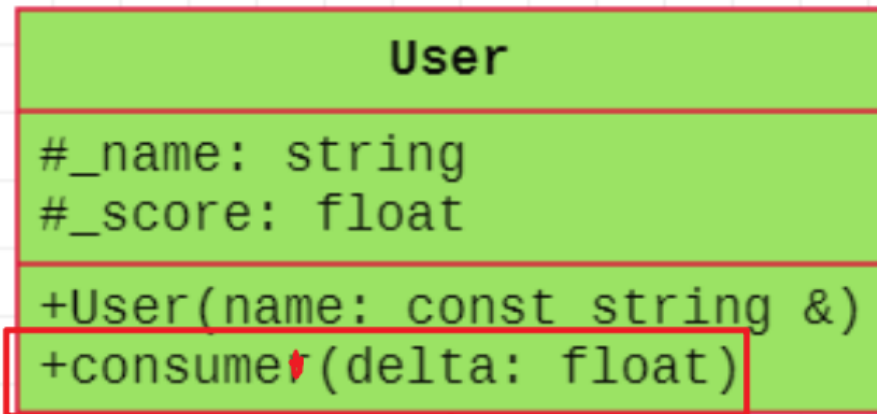


3、里氏替换原则（重要）

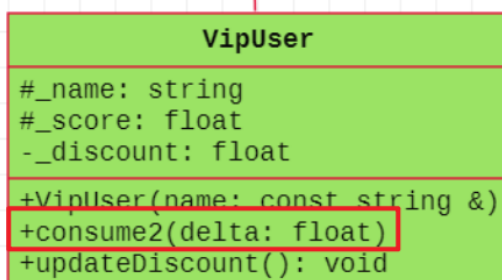
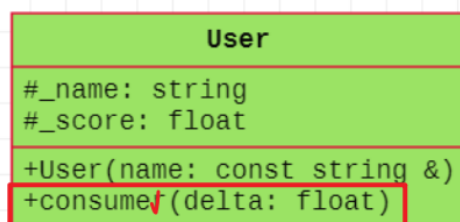
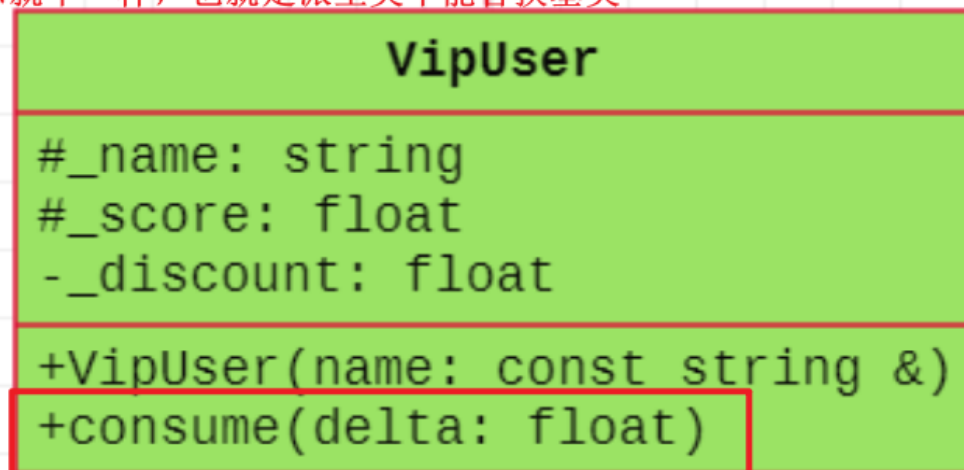
核心思想是：派生类必须能够替换其基类。

可以表现为：

- 派生类可以实现基类的抽象方法，体现多态（正好就是C++中多态的含义）
- 派生类可以增加新的个性/功能（C++中的新增功能的特点）
- 派生类不能覆盖基类中的非虚函数（C++中的隐藏）（基类与派生类中对于非虚函数而言，不要同名）



对于基类与派生类而言，都有 **consume** 函数，该函数都是非虚函数，所以在C++中会存在隐藏的概念，那么派生类与基类对象调用该函数的时候，体现的形态就不一样，也就是派生类不能替换基类

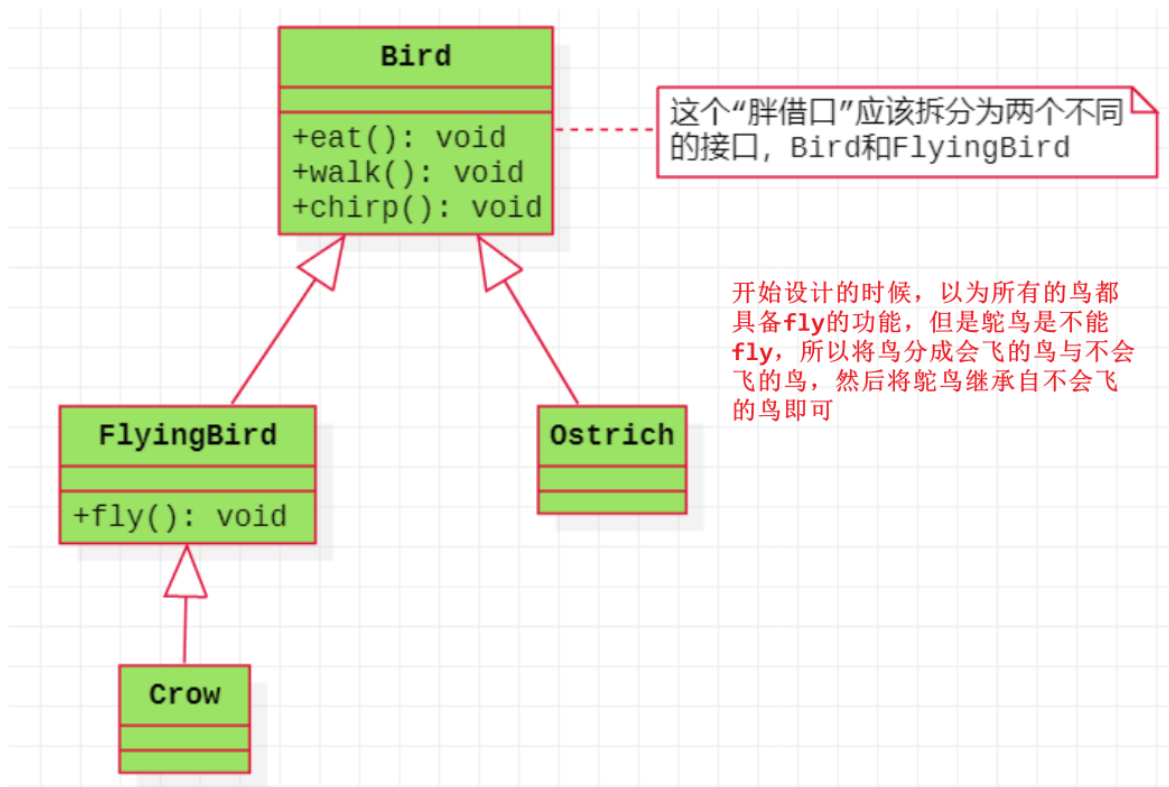


为了解决C++中隐藏导致的问题，所以基类与派生类中对于非虚函数而言，不要同名即可

派生类不能直接实现consumer方法，而是在自己实现一个方法consume2

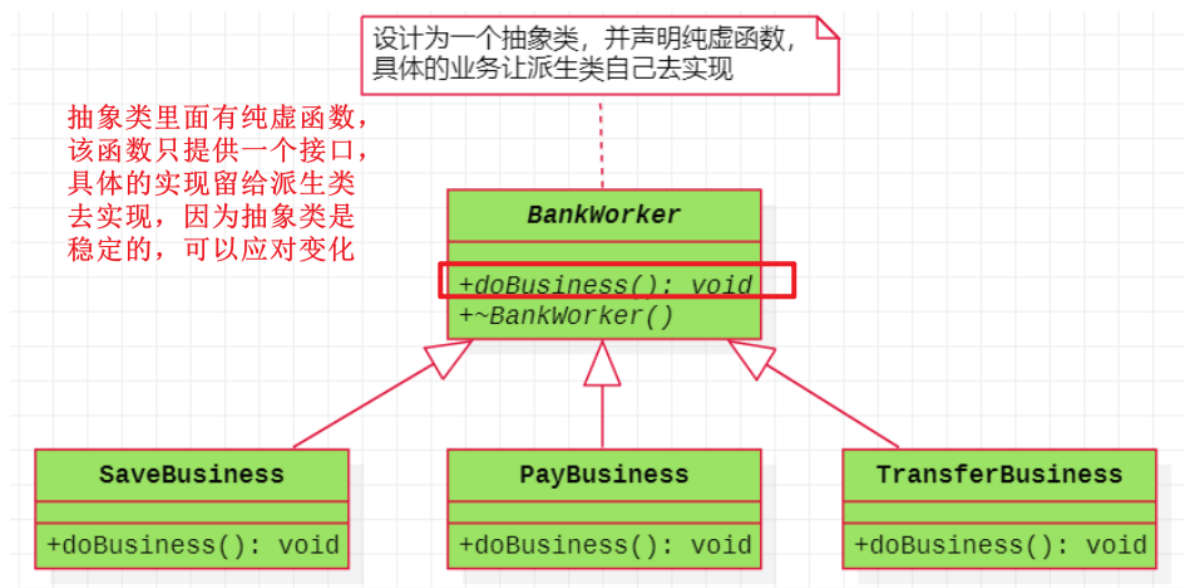
4、接口分离原则

核心思想：使用多个小的专门的接口，而不要使用一个大的总接口



5、依赖倒置原则 (重要)

核心思想：面向接口编程，依赖于抽象(抽象是稳定的，具体的是在变化的)

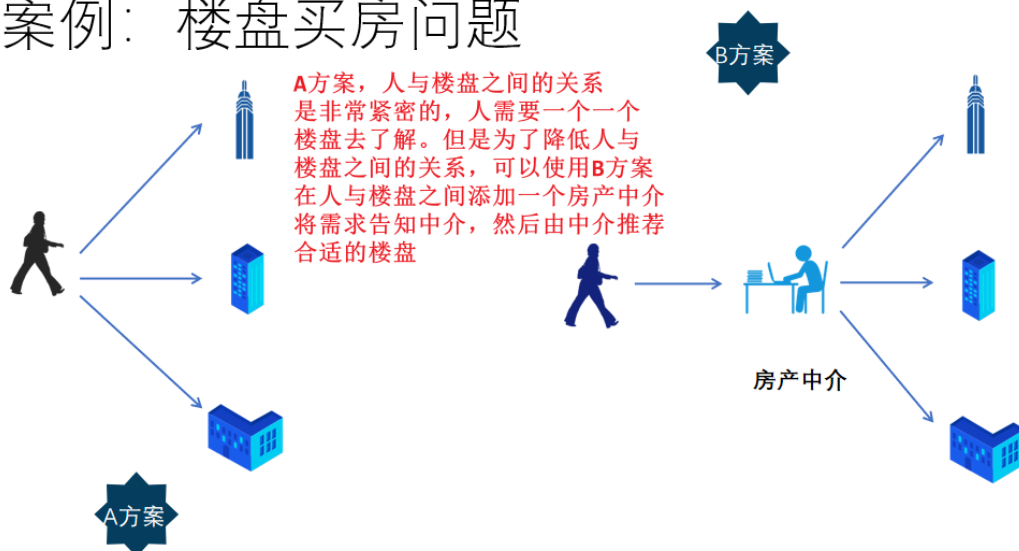


在大多数情况下，开闭原则、里氏代换原则和依赖倒置原则会同时出现，**开闭原则是目标，里氏代换原则是基础，依赖倒置原则是手段**

6、最少知识 (知道) 原则

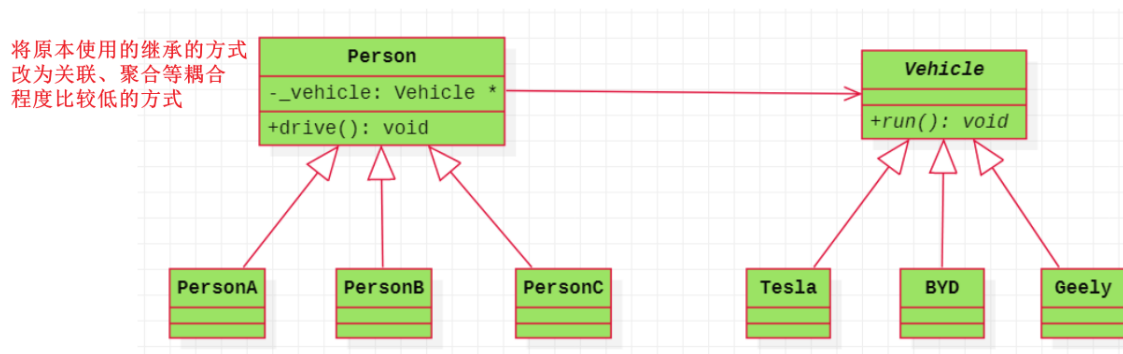
减少类与类、模块与模块之间的耦合程度。

案例：楼盘买房问题



7、组合复用原则

使用关联、聚合取代继承关系。



四、设计模式

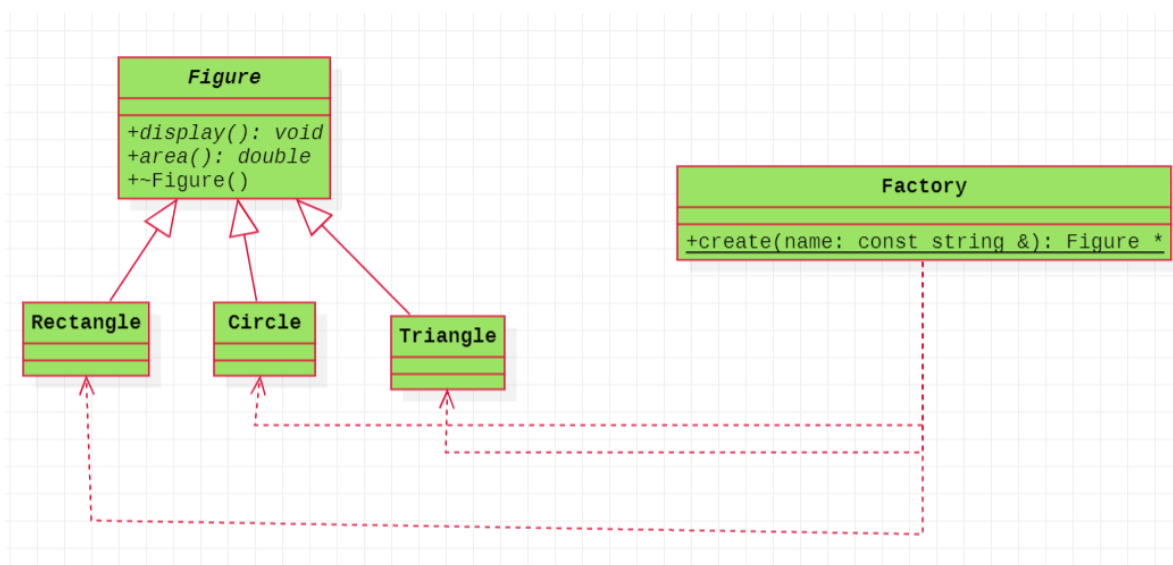
1、概念

它是解决特定问题的一系列套路，有一定的普遍性。它是一套被反复使用、多数人知晓的、代码设计经验的总结。它可以提高代码的可重用性、代码的可读性和代码的可靠性以及可扩展性。

设计模式有23种。分类创建型、结构型、行为型。

2、简单工厂

2.1、类图



2.2、代码

```

131 //该模式可以根据产品的名字生产出对应的产品
132 class Factory
133 {
134 public:
135     static Figure *create(const string &name)
136     {
137         if(name == "rectangle")    根据产品的名字生产产品
138         {
139             //配置文件进行传递进来
140             //txt、xml、yang
141             //读配置文件
142             //获取配置文件中的信息
143             Rectangle *prec = new Rectangle(10, 20);
144
145             return prec;
146         }
147         else if(name == "circle")
148         {
149             //配置文件进行传递进来
150             //txt、xml、yang
  
```

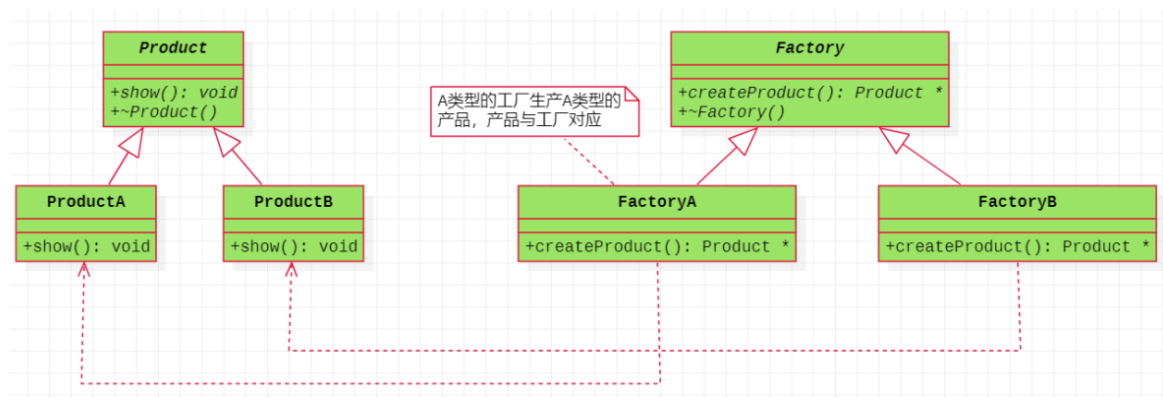
2.3、优缺点

```

124 //简单工厂（静态工厂）
125 //缺点：
126 //1、违反了单一职责原则
127 //2、违反了开放闭合原则
128 //3、违反了依赖倒置原则
129 //
130 //优点
131 //该模式可以根据产品的名字生产出对应的产品
  
```

3、工厂方法

3.1、类图



3.2、代码

```

124 //工厂方法（工厂模式）
125 //会为每个产品创建对应的工厂，那么工厂的数量会急剧上升
126 class Factory
127 {
128 public:
129     virtual Figure *create() = 0;
130     virtual ~Factory(){}
131 };
132
133 class RectangleFactory 长方形的产品创建长方形工厂
134 : public Factory
135 {
136 public:
137     virtual Figure *create() override
138     {
139
140
141
142
143
144
145
146
147
148
149 class CircleFactory 圆形产品创建圆形工厂
150 : public Factory
151 {
152 public:
153     virtual Figure *create() override
154     {
155
156
157
158
159
160
161
162
163
164
165 class TriangleFactory 三角形产品创建三角形工厂
166 : public Factory
167 {
168 public:
169     virtual Figure *create() override
170     {
  
```

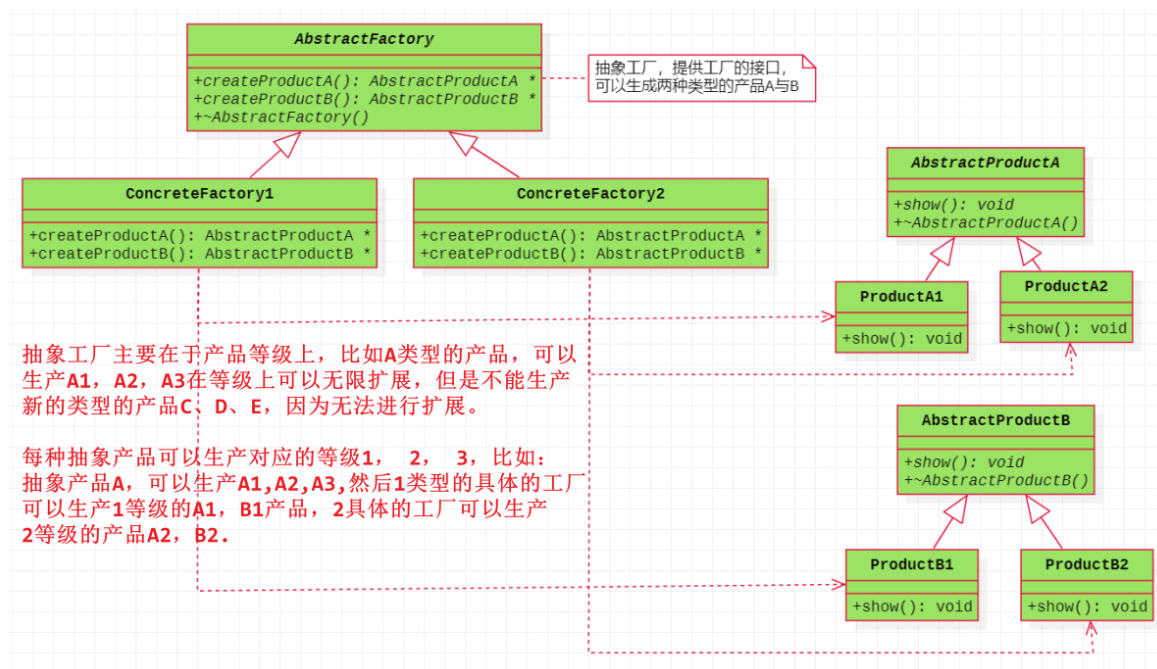
3.3、优缺点

```

124 //工厂方法（工厂模式）
125 //会为每个产品创建对应的工厂，那么工厂的数量会急剧上升
  
```

4、抽象工厂

4.1、类图



Day02

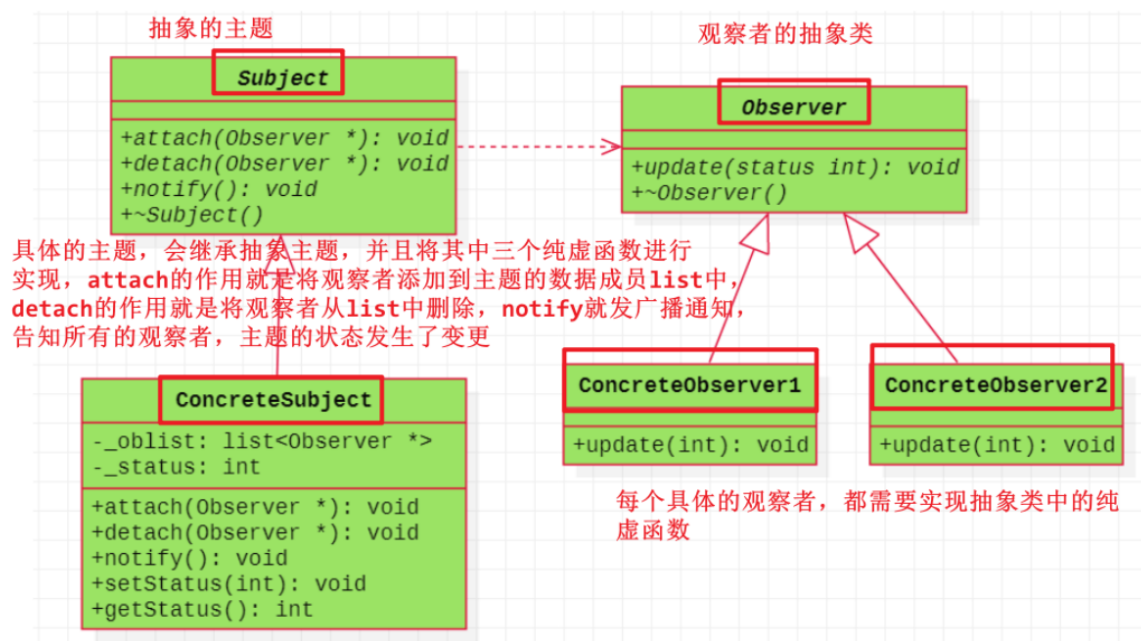
一、问题回顾

- 1、类与类之间的关系有哪几种？有什么特点？
- 2、面向对象的设计原则有哪几种？每种的核心思想是什么？
- 3、什么是设计模式？有多少种设计模式？分为哪三种类型？

二、观察者模式

定义对象的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新

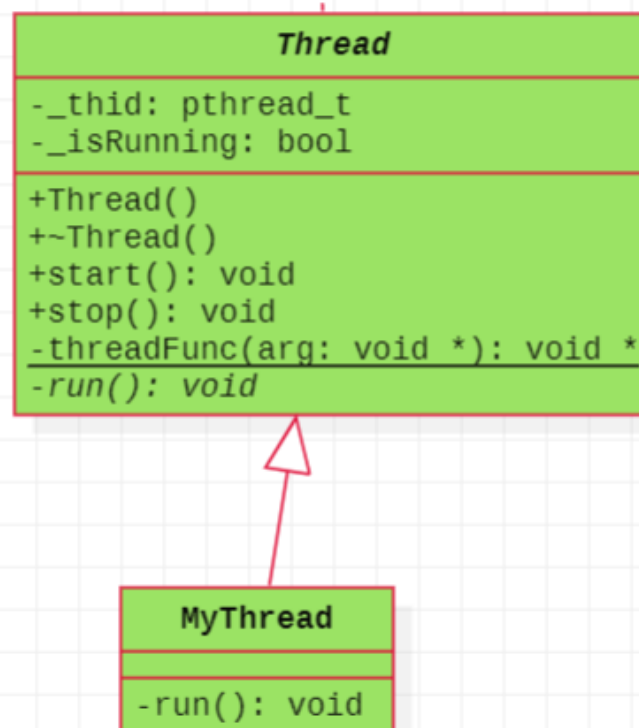
类图



三、面向对象的线程封装

1、类图

将线程封装成一个类，为了唯一标识每个线程，所以设置数据成员`_thid`。为了记录线程是否在运行，所以增加了`_isRunning`标志位。构造函数与析构函数的作用是为了给数据成员进行初始化与清理，但是该类中，数据成员是内置类型，所以无需清理。线程要创建并且运行，所以设置了`start`函数，在该函数中调用`pthread_create`函数，而该函数的第三个参数会调用线程入口函数，就添加`threadFunc`函数，但是并不知道该函数具体要执行什么任务，那么就将任务留给派生类使用，所以就增加了纯虚函数`run`方法。线程要停止，所以增加了`stop`函数，如果标志位是`true`，就可以在`stop`中调用`pthread_join`函数。



2、几个问题

2.1、将threadFunc函数设置为静态的

```
20 //线程的启动
21 void Thread::start()
22 {
23     //shift + k
24     //为了消除threadFunc的this问题，需要将其设置为static
25     /* int ret = pthread_create(&_thid, nullptr, threadFunc, nullptr); */
```

2.2、run方法的调用

```

20 //线程的启动
21 void Thread::start()
22 {
23     //shift + k
24     //为了消除threadFunc的this问题，需要将其设置为static
25     /* int ret = pthread_create(&_thid, nullptr, threadFunc, nullptr); */
26     int ret = pthread_create(&_thid, nullptr, threadFunc, this);
27     if(ret)
28
29
30
31
32 //线程入口函数
33 void *Thread::threadFunc(void *arg)
34 {
35     //pth = arg = new MyThread()
36     Thread *pth = static_cast<Thread *>(arg);
37     //Thread * this
38     if(pth)
39     {
40         pth->run(); //会体现多态
41     }
42 }

```

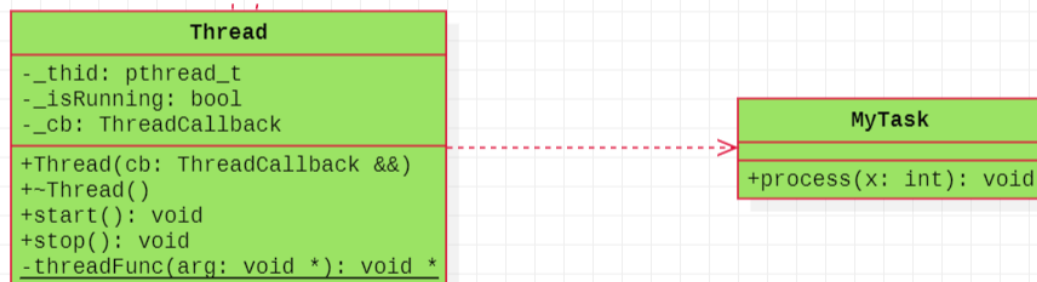
Day03

一、基于对象的线程

1、类图

将线程封装成一个类，为了唯一标识每个线程，所以设置数据成员_thid
 为了记录线程是否在运行，所以增加了_isRunning标志位。
 构造函数与析构函数的作用是为了给数据成员进行初始化与清理，但是该类中，数据成员是内置类型，所以无需清理。线程要创建并且运行，所以设置了start函数，在该函数中调用pthread_create函数，而该函数的第三个参数会调用线程入口函数，就添加threadFunc函数，但是并不知道该函数具体要执行什么任务，那么就将任务留给派生类使用，所以就增加了纯虚函数run方法。线程要停止，所以增加了stop函数，如果标志位是true，就可以在stop中调用pthread_join函数

```
using ThreadCallback = function<void*>
```



2、几个问题

2.1、新增数据成员的设计

```

9  class Thread
10 {    将线程要做的任务使用function包装
11     using ThreadCallback = function<void()>;
12 public:
13     /* Thread(const ThreadCallback &cb); */
14     Thread(ThreadCallback &&cb);
15
16 private:
17     pthread_t _thid; //线程id
18     bool isRunning; //标识线程是否运行的标志
19     ThreadCallback _cb; //

```

2.2、回调函数的注册

```

17 Thread::Thread(ThreadCallback &&cb)
18 : _thid(0)
19 , isRunning(false)
20 , _cb(std::move(cb)) //注册
21 {
22
23 }

```

2.3、回调函数的执行

```

--
62 //线程入口函数
63 void *Thread::threadFunc(void *arg)
64 {
65     //pth = arg = &mth
66     Thread *pth = static_cast<Thread *>(arg);
67     //Thread * this
68     if(pth)
69     {
70         pth->_cb(); //回调
71     }
72     else
73     {
74         cout << "nullptr == pth" << endl;
75     }
76
77     /* return nullptr; */
78     pthread_exit(nullptr);
79 }

```

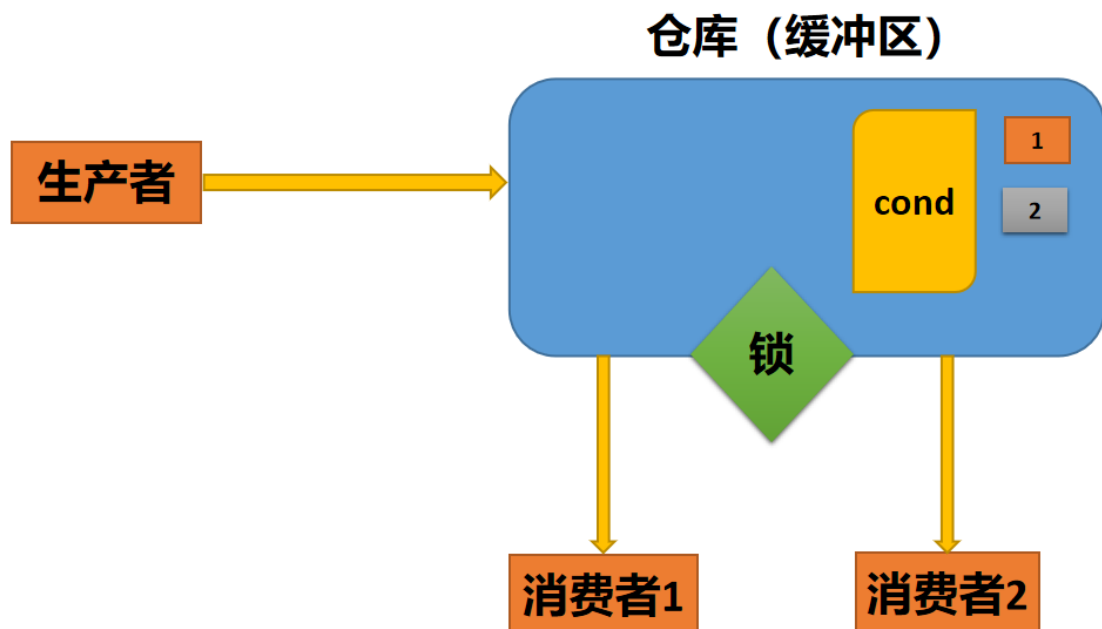
2.4、任务的传递

```
11 class MyTask
12 {
13 public:
14     void process(int x)
15     {
16         //...实现具体的逻辑
17         while(1)
18         {
19             cout << "MyTask is running!!!" << endl;
20             sleep(x);
21         }
22     }
23
24 void test()
25 {
26     MyTask task; // 栈对象
27     function<void()> f = bind(&MyTask::process, &task, 1);
28     Thread mth(std::move(f));
29     /* Thread mth(bind(&MyTask::process, &task, 1)); */
30     mth.start();
31     mth.stop();
32 }
33
```

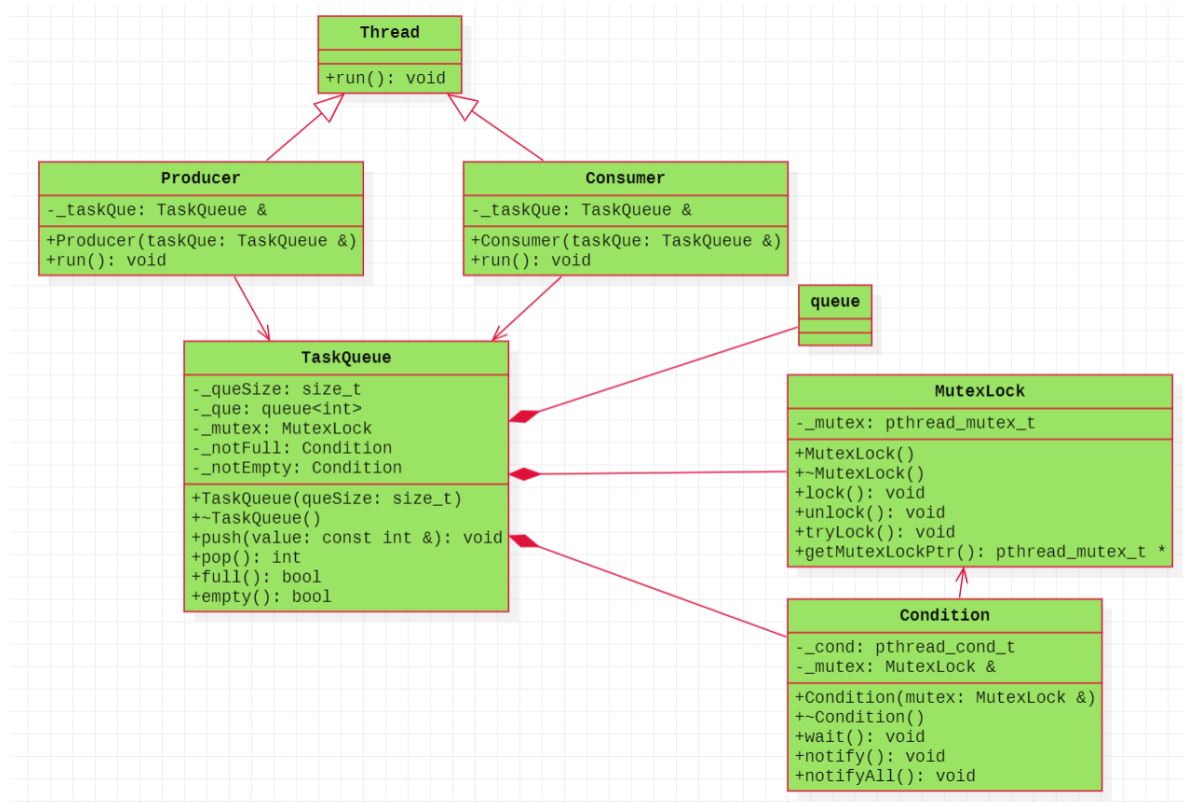
使用bind改变process函数的形态
然后传递给线程Thread

二、面向对象的PC

1、原理图



2、类图



3、几个问题

3.1、RAII思想防止死锁

```

19 void TaskQueue::push(const int &value)
20 {
21     //使用C++之父提出的RAII的思想
22     //核心：利用栈对象的生命周期管理资源
23     //1、上锁
24     /* _mutex.lock(); */
25     MutexLockGuard autoLock(_mutex);
26
27     //2、是不是满
28     /* if(full()) */
29     while(full())
30     {
31         //2.1、如果队列是满的，生产者需要睡眠
32         _notFull.wait();
33     }
34     //2.2、如果队列不为满才能进行push操作
35     _que.push(value);
36     //并且将消费者唤醒
37     _notEmpty.notify();
38
39     //3、解锁
40     /* mutex.unlock(); */
  
```

```

27 class MutexLockGuard
28 {
29 public:
30     MutexLockGuard(MutexLock &mutex)
31     : _mutex(mutex)
32     {
33         _mutex.lock(); 构造函数中上锁
34     }
35
36     ~MutexLockGuard()
37     {
38         _mutex.unlock(); 析构函数中解锁
39     }
40 private:
41     MutexLock &_mutex;
42 };

```

3.2、虚假唤醒

```

19 void TaskQueue::push(const int &value)
20 {
21     //使用C++之父提出的RAII的思想
22     //核心：利用栈对象的生命周期管理资源
23     //1、上锁
24     /* _mutex.lock(); */
25     MutexLockGuard autoLock(_mutex);
26
27     //2、是不是满
28     /* if(full()) */
29     while(full()) 将if改为while
30     {
31         //2.1、如果队列是满的，生产者需要睡眠
32         _notFull.wait();
33     }
34     //2.2、如果队列不为满才能进行push操作
35     _que.push(value);
36     //并且将消费者唤醒
37     _notEmpty.notify();
38
39     //3、解锁
40     /* _mutex.unlock(); */

```

3.3、禁止复制

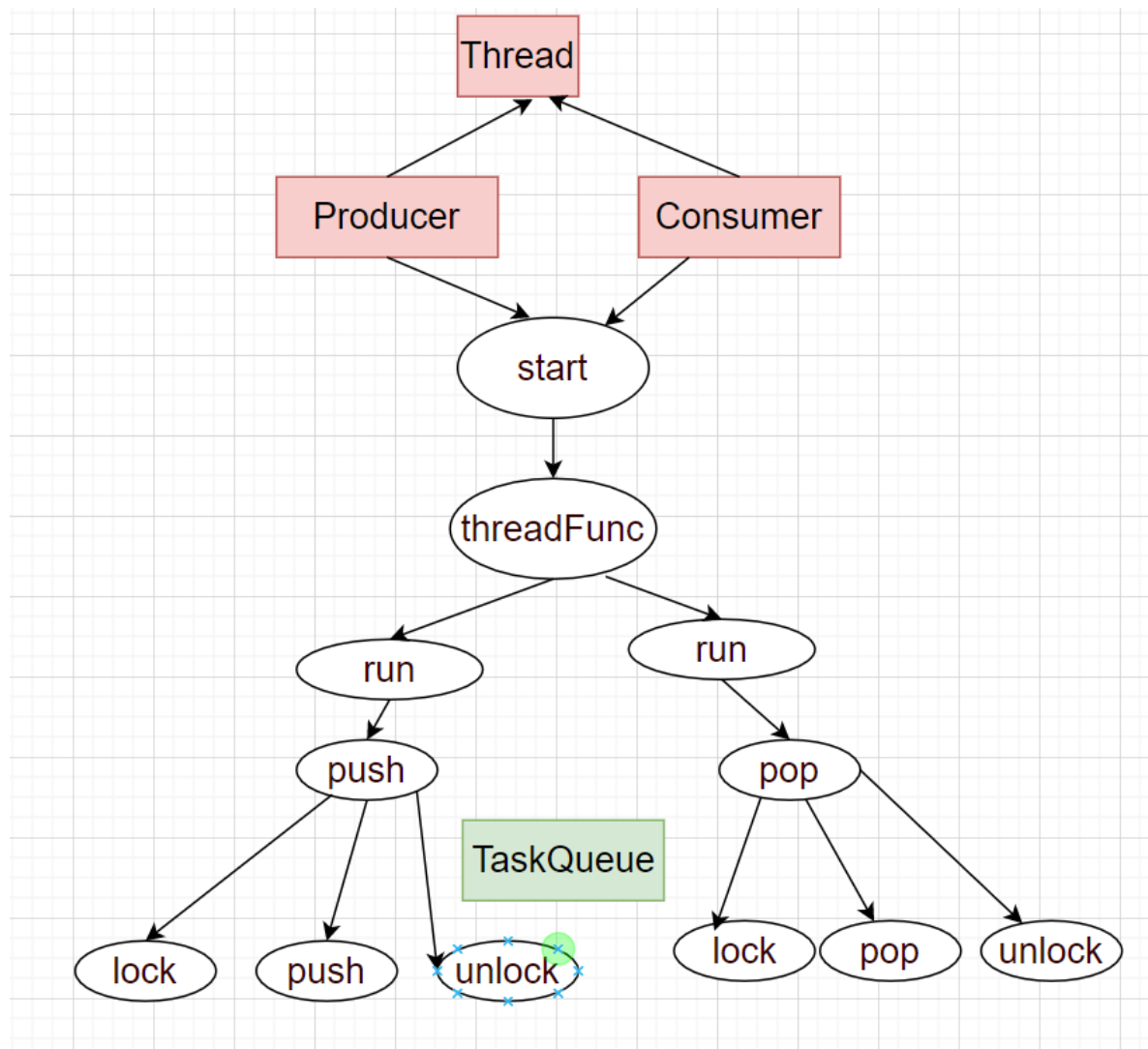
```

4 class NonCopyable
5 {
6     /* public: */
7     protected:
8         NonCopyable() = default;
9         NonCopyable(const NonCopyable &rhs) = delete;
10        NonCopyable &operator=(const NonCopyable &rhs) = delete;
11
12 private:
13
14 };
15
16 class MutexLock
17 : NonCopyable
18 {
19 public:
20     MutexLock();
21     ~MutexLock();

```

将基类的拷贝构造函数与赋值运算符函数删除

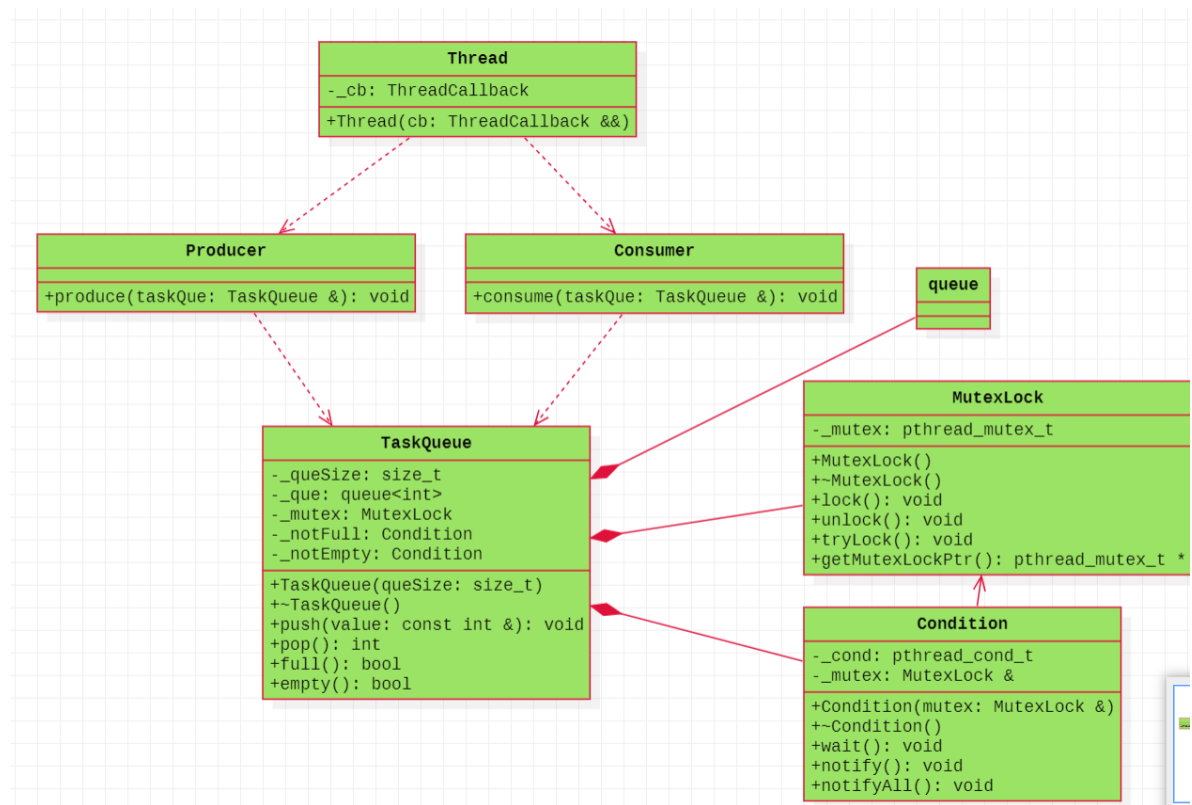
4、代码流程图



Dya04

一、基于对象的PC封装

1、类图



2、几个问题

2.1、生产者代码

```

20
21 void Producer::produce(TaskQueue &taskQue) taskQueue作为参数传递进来
22 {
23     ::srand(::clock()); //种随机种子
24     int cnt = 20;
25     while(cnt--)
26     {
27         int number = ::rand() % 100; //产生随机数
28         taskQue.push(number);
29         cout << ">>B0.Producer produce number = " << number << endl;
30         sleep(1);
31     }
32 }
  
```

2.2、消费者代码

```

10
19 void Consumer::consume(TaskQueue &taskQue) taskQueue作为参数传递进来
20 {
21     int cnt = 20;
22     while(cnt--)
23     {
24         int number = taskQue.pop();
25         cout << "<<B0.Consumer consume number = " << number << endl;
26         sleep(1);
27     }
28 }
  
```

2.3、测试代码

```

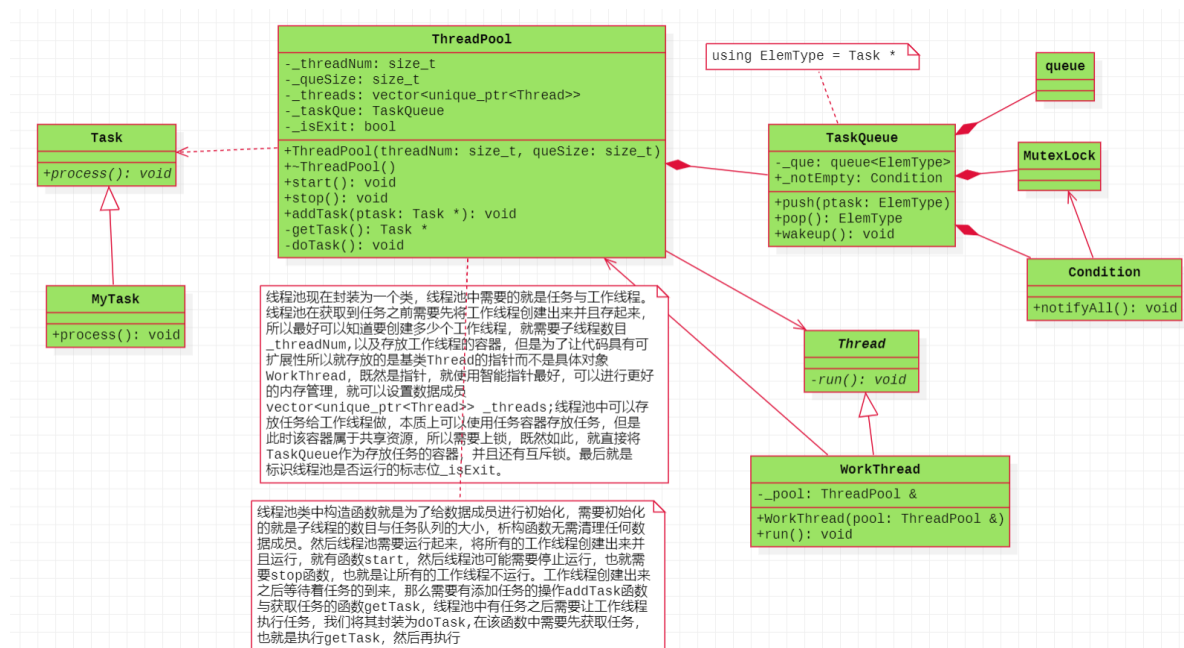
12 void test()
13 {
14     TaskQueue taskQue(10);
15
16     Producer producer;
17     Consumer consumer;
18
19     unique_ptr<Thread> pro(new Thread(bind(&Producer::produce, &producer, std::ref(taskQue))));
20     unique_ptr<Thread> con(new Thread(bind(&Consumer::consume, &consumer, std::ref(taskQue))));
21
22     pro->start();
23     con->start();
24
25     pro->stop();
26     con->stop();
27 }

```

使用bind改变函数形态，同时注意bind默认使用的是值传递，为了引用传递，可以使用std::ref

二、面向对象的线程池封装

1、类图



2、几个问题

2.1、线程池中数据成员

```

34 private:
35     size_t _threadNum; //子线程的数目
36     size_t _queSize; //任务队列的大小
37     vector<unique_ptr<Thread>> _threads; //存放工作线程的容器
38     TaskQueue _taskQue; //存放任务的容器
39     bool _isExit; //标志线程池是否结束的标志位
40 }

```

2.2、线程池中任务执行不完

```

44 void ThreadPool::stop()
45 {
46     //任务如果执行不完，就不能让子线程退出
47     while(!_taskQue.empty())
48     {
49         sleep(1);
50     }
51
52     _isExit = true;
53
54     //在回收之前将所有的工作线程唤醒
55     _taskQue.wakeup();
56
57     //将工作线程停止
58     for(auto &th : _threads)
59     {
60         th->stop();
61     }
62 }

```

如果缺少这段代码，有可能任务没有执行完毕，程序就退出来了，现在任务没有执行完毕就无法继续下面的步骤

2.3、线程池无法退出原因

```

79 void ThreadPool::doTask()
80 {
81     //用的是线程池是否退出的标志位???
82     //子线程（工作线程）在获取完任务之后，那么任务队列就为空
83     //接下来子线程会执行任务，也就是执行process函数，而主线程
84     //会执行stop函数中的将标志位_isExit设置为true的操作。如果
85     //子线程执行process的速率非常慢，那么当子线程执行任务之前
86     //主线程已经将标志位设置为了true，那么子线程就不会进入到
87     //while循环中，那么程序就可以正常结束；如果子线程执行任务
88     //process的速率非常的快，但是主线程还没有将标志位设置为
89     //true，那么子线程执行完process后，又进入了while循环，在
90     //执行getTask的时候，也就是会执行TaskQueue中的pop，然后
91     //睡眠
92     while(!_isExit)
93     {
94         Task *ptask = getTask();
95         if(ptask)
96         {
97             ptask->process();//执行任务
98             /* sleep(3); */
99         }

```

2.4、解决线程池无法退出的方法

```

44 void ThreadPool::stop()
45 {
46     //任务如果执行不完，就不能让子线程退出
47     while(!_taskQue.empty())
48     {
49         sleep(1);
50     }
51
52     _isExit = true;
53
54     //在回收之前将所有的工作线程唤醒
55     _taskQue.wakeup();
56
57     //将工作线程停止
58     for(auto &th : _threads)
59     {
60         th->stop();
61     }
62 }

```

但是又会出现新的问题，就是TaskQueue中wait函数在while循环中。

```

73 //将所有等待在_notEmpty条件变量上的线程唤醒
74 void TaskQueue::wakeup()
75 {
76     _flag = false;
77     _notEmpty.notifyAll();
78 }

```

```
38 ElemType TaskQueue::pop()
39 {
40     MutexLockGuard autoLock(_mutex);
41
42     while(empty() && _flag)
43     {
44         _notEmpty.wait();
45     }
46
47     if(_flag)
48     {
49         ElemType tmp = _que.front();
50         _que.pop();
51         _notFull.notify();
52
53         return tmp;
54     }
55     else
56     {
57         return nullptr;
58     }
59
```

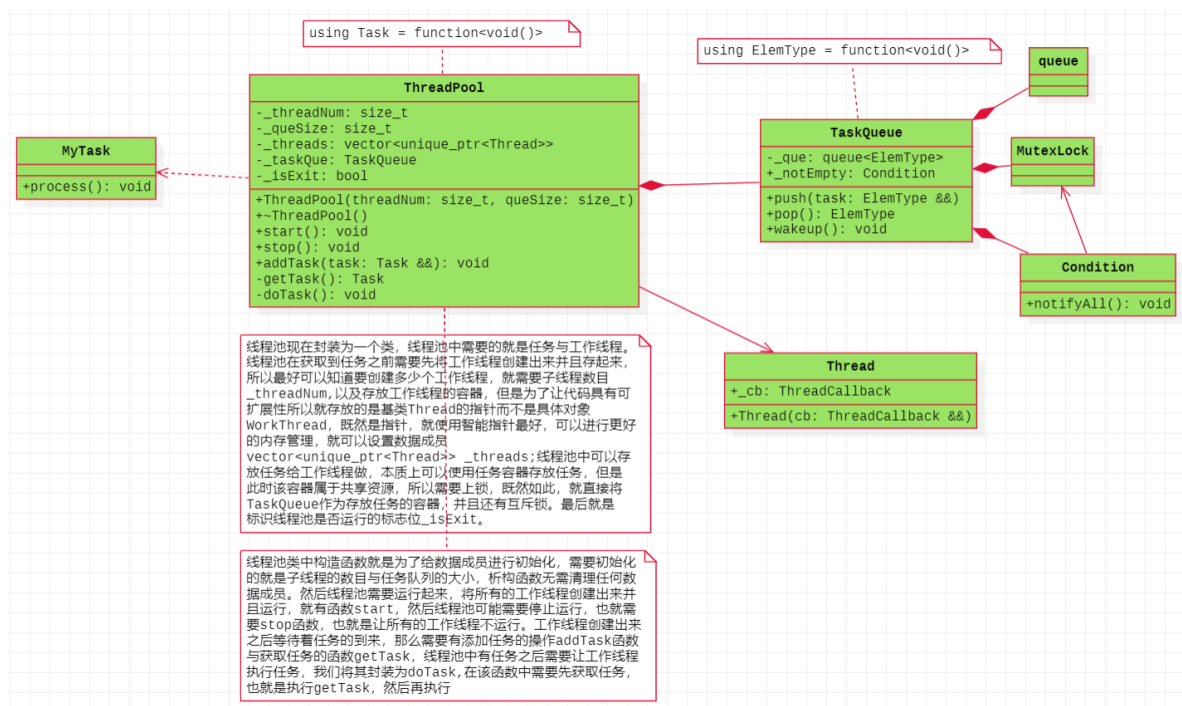
增加标志位，让子线程
可以继续后走

3、序列图

见UML

三、基于对象线程池封装

1、类图



2、几个问题

2.1、工作线程获取任务

```

--
24 //线程池的启动与停止
25 void ThreadPool::start()
26 {
27     //创建工作线程
28     for(size_t idx = 0; idx != _threadNum; ++idx)
29     {
30         _threads.push_back(unique_ptr<Thread>(new Thread(bind(&ThreadPool::doTask, this))));
31     }
32     //将工作线程运行起来
33     for(auto &th : _threads)
34     {
35         th->start();
36     }
37 }
38 }

```

给每个工作线程传递线程池交给的任务doTask，然后交给Thread的数据成员_cb

2.2、向线程池添加任务

```

9 class MyTask
10 {
11 public:
12     void process() 真正需要执行的任务
13     {
14         ::srand(::clock()); //种随机种子
15         int number = ::rand() % 100; //产生随机数
16         cout << ">>BO.ThreadPool.MyTask number = " << number << endl;
17     }
18 };

```

```

20 void test()
21 {
22     unique_ptr<MyTask> ptask(new MyTask());
23     ThreadPool pool(4, 10);
24     pool.start();
25
26     int cnt = 20;
27     while(cnt-->0)
28     {
29         //添加任务 将MyTask中的process函数添加到线程池中
30         pool.addTask(std::bind(&MyTask::process, ptask.get()));
31         cout << "cnt = " << cnt << endl;
32     }
33
34     pool.stop();
35 }

```

2.3、获取任务并且执行

```

88     while(!_isExit)
89     {
90         Task taskcb = getTask(); 获取任务
91         if(taskcb)
92         {
93             taskcb();//回调 这个其实就是添加进来的MyTask中的
94             //process函数
95         }
96         else
97         {
98             cout << "nullptr == ptask" << endl;
99         }
100     }

```

Day05

一、C++11的线程

1、对象的创建

```

1 | template< class Function, class... Args >
2 | explicit thread( Function&& f, Args&&... args )

```

2、线程的退出

```

1 | void join();

```

3、获取线程id

```

1 | std::thread::id get_id() const noexcept

```

可以使用线程对象调用成员函数get_id，获取线程的id

但是一般使用的是

```
1 | std::thread::id get_id() noexcept;
```

4、线程入口函数的传递方式 (重要)

4.1、普通函数的形式

```
8 void threadFunc(int x)
9 {
10     cout << "void threadFunc(int)" << endl;
11     cout << "x = " << x << endl;
12     cout << "child thread id = " << std::this_thread::get_id() <<
13 }
14
15 int main(int argc, char *argv[])
16 {
17     cout << "main thread id = " << std::this_thread::get_id() <<
18     /* test(); */
19     /* int a = 1; */
20     thread th(threadFunc, 1);
21     /* cout << "child thread = " << th.get_id() << endl; */
22     th.join();
23
24     return 0;
25 }
```

将线程入口函数以普通函数的形式进行传递

4.2、函数指针 (引用) 的形式

```
8 void threadFunc(int x)
9 {
10     cout << "void threadFunc(int)" << endl;
11     cout << "x = " << x << endl;
12     cout << "child thread id = " << std::this_thread::get_id() <<
13 }
14
15 int main(int argc, char *argv[])
16 {
17     cout << "main thread id = " << std::this_thread::get_id() <<
18     typedef void (*pFunc)(int);
19     pFunc f = &threadFunc;
20
21     thread th(f, 2);
22     th.join();
23
24     return 0;
25 }
```

2、线程入口函数传递函数指针

4.3、函数对象的形式

```

8 class Example
9 {
10 public:
11     void operator()(int x)
12     {
13         cout << "void operator()(int)" << endl;
14         cout << "x = " << x << endl;
15         cout << "child thread id = " << std::this_thread::get_id() << endl;
16     }
17 };
18
19 int main(int argc, char *argv[])
20 {
21     cout << "main thread id = " << std::this_thread::get_id() << endl;
22
23     Example ex;
24     /* ex(3); */
25
26     thread th(ex, 3); 3、线程入口函数传递函数对象
27     th.join();
28

```

4.4、传递lambda表达式

```

8 int main(int argc, char *argv[])
9 {
10     cout << "main thread id = " << std::this_thread::get_id() << endl;
11
12     thread th([](int x){ 4、线程入口函数传递lambda表达式
13         cout << "x = " << x << endl;
14         cout << "child thread id = " << std::this_thread::get_id()
15         }, 4);
16     th.join();
17
18     return 0;
19 }
20

```

4.5、传递function对象

```

11 void threadFunc(int x)
12 {
13     cout << "void threadFunc(int)" << endl;
14     cout << "x = " << x << endl;
15     cout << "child thread id = " << std::this_thread::get_id() << endl;
16 }
17
18 int main(int argc, char *argv[])
19 {
20     cout << "main thread id = " << std::this_thread::get_id() << endl;
21
22     using namespace std::placeholders;
23     function<void(int)> f = bind(threadFunc, _1);
24
25     thread th(f, 6); 5、线程入口函数传递function类型
26     th.join();
27

```

二、互斥锁

1、锁mutex

```

9
10 mutex mtx;    C++11提供的互斥锁
11 int gCnt = 0;
12
13 void threadFunc()
14 {
15     for(int idx = 0; idx < 10000000; ++idx)
16     {
17         mtx.lock();    上锁操作
18         ++gCnt;
19         mtx.unlock();  解锁操作
20     }
21 }

```

2、锁lock_guard

```

11 mutex mtx;
12 int gCnt = 0;
13
14 void threadFunc()
15 {
16     for(int idx = 0; idx < 10000000; ++idx)
17     {
18         lock_guard<mutex> lg(mtx);
19         ++gCnt;    C++11提供的锁，利用RAII的思想，
20         //...      利用栈对象的生命周期管理资源，
21         //...      在构造函数中上锁，在析构函数
22         //xxxxx    中解锁。
23         //yyyyy
24     }
25 }

```

3、锁unique_lock

```

11 mutex mtx;
12 int gCnt = 0;
13
14 void threadFunc()
15 {
16     for(int idx = 0; idx < 10000000; ++idx)
17     {
18         unique_lock<mutex> ul(mtx);
19         ++gCnt;
20         //...
21         //...
22         /* ul.unlock(); */
23         //xxxxx
24         /* ul.lock(); */
25         //yyyyy
26         /* ul.unlock(); */
27         //zzzz
28     }
29 }

```

C++11提供的锁，使用了RAII思想，利用栈对象的生命周期管理资源，在构造函数中初始化资源（加锁），在析构函数中释放资源（解锁）。同时比lock_guard更加灵活，可以认为手动加锁与解锁，还可以配合与条件变量一起进行使用

三、条件变量

1、条件变量condition_variable

```

27 private:
28     size_t _queSize; //任务队列的大小
29     queue<int> _que; //存放任务的数据结构
30     mutex mutex; //互斥锁
31     condition_variable _notFull; //非满条件变量（生产者）
32     condition_variable _notEmpty; //非空条件变量（消费者）
33 };

```

```

18 //添加任务
19 void TaskQueue::push(const int &value)
20 {
21     unique_lock<mutex> autoLock(_mutex);
22
23     while(full())
24     {
25         _notFull.wait(autoLock);
26     }
27     _que.push(value);
28     _notEmpty.notify_one();
29 }

```

生产者消费者代码中，直接使用条件变量的wait函数，但是需要配合unique_lock对象，同时也可以使用条件变量的通知函数notify_one或者notify_all

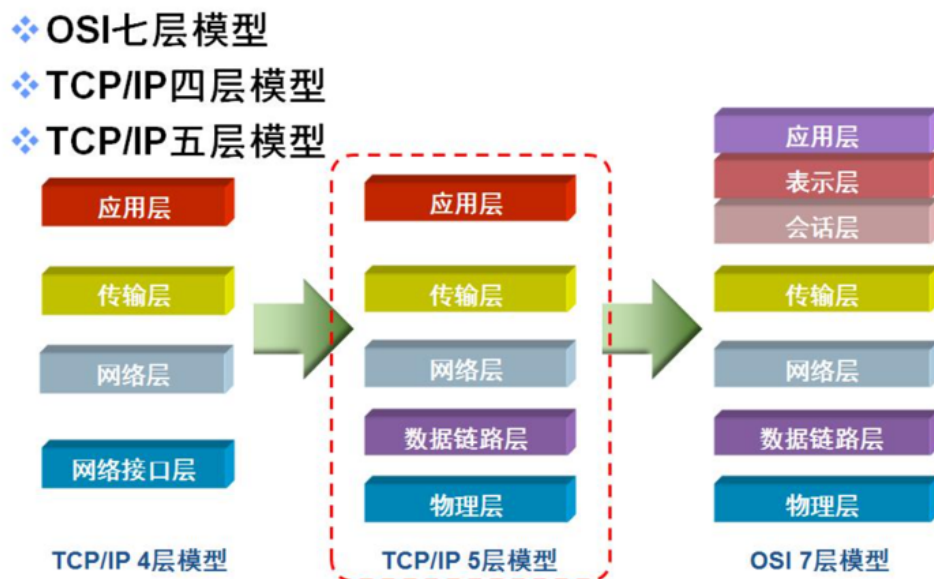
四、计算机网络基础

1、协议

通信双方遵循的规则。

2、几层协议

七层协议 vs 四(五)层协议

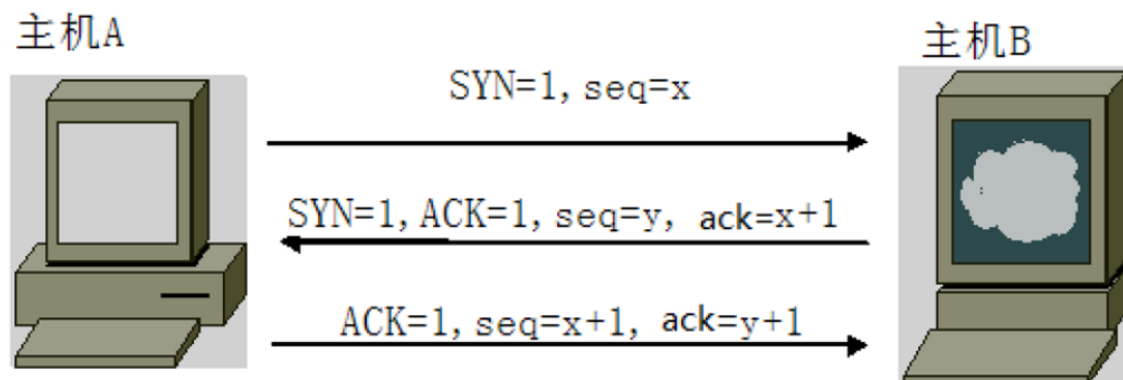


3、TCP协议 (重要)

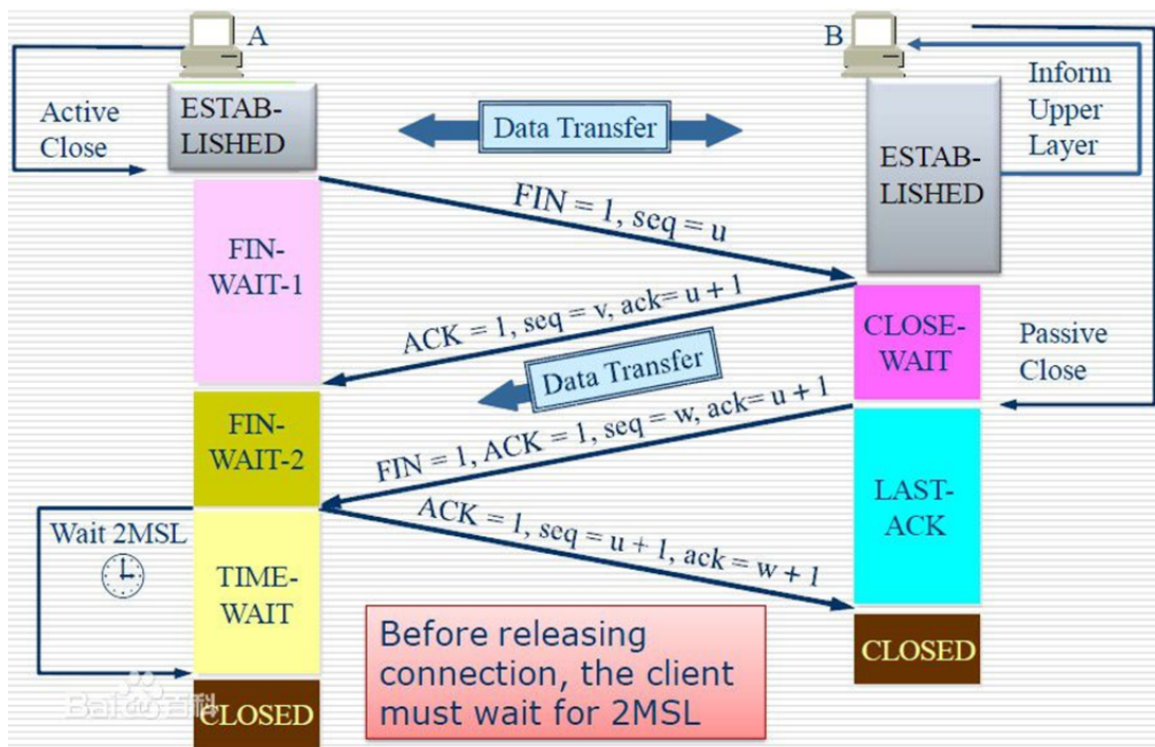
协议的头能看懂就可以了。

TCP协议是一个传输层的协议、连接的协议、可靠的协议、全双工的协议、字节流的协议；TCP是一个可以进行流量控制的协议。

三次握手

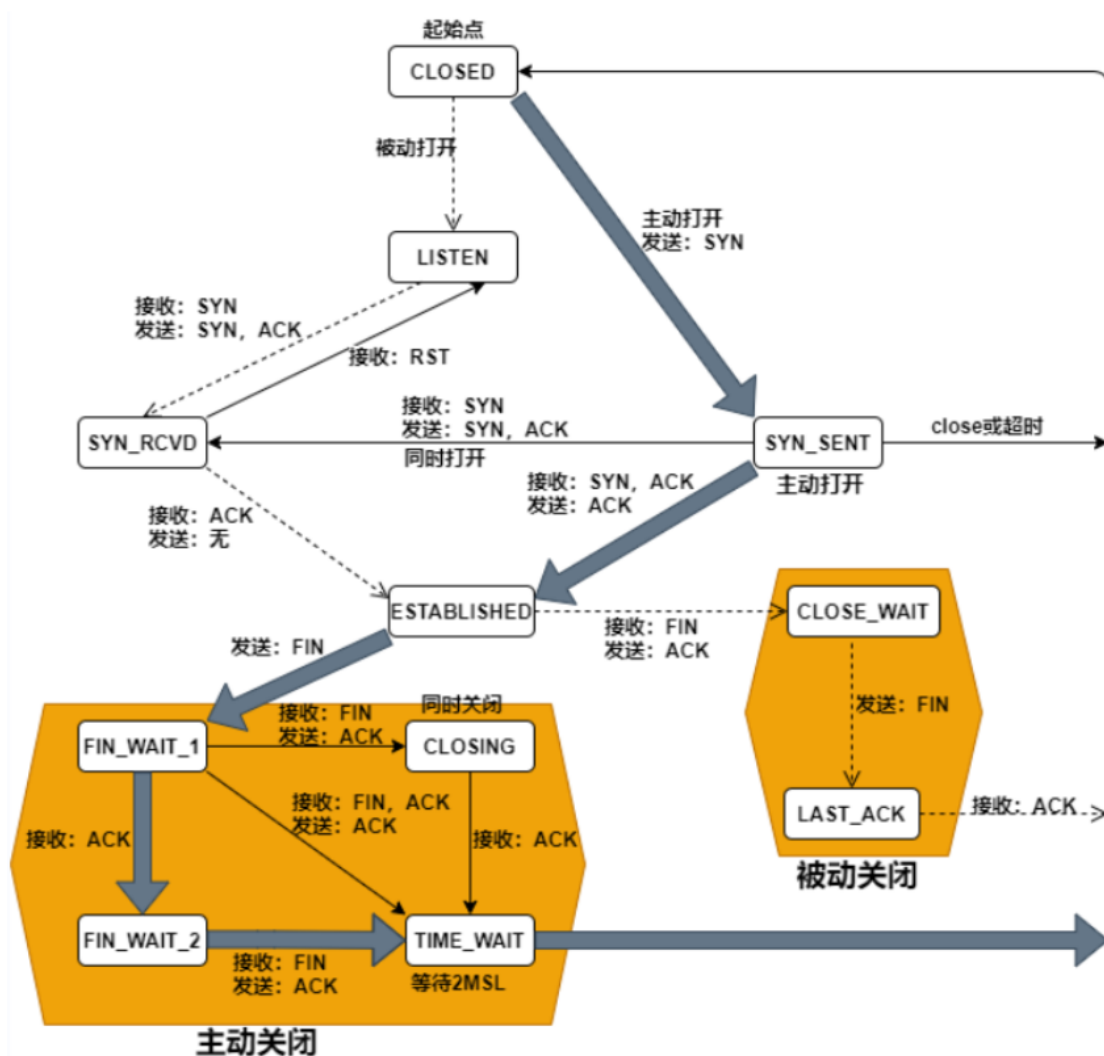


四次挥手



4、状态迁移图 (重要)

11种状态。



```
wangdao@wangdao:6_Socket$
wangdao@wangdao:6_Socket$netstat -apn | grep 8888
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
tcp        0      0 127.0.0.1:8888      0.0.0.0:*           LISTEN      11902/./server
tcp        0      0 127.0.0.1:8888      127.0.0.1:49368     ESTABLISHED 11902/./server
tcp        0      0 127.0.0.1:49368     127.0.0.1:8888     ESTABLISHED 11905/./client
```

五、网络编程

1、基础

在网络环境中要确定一台主机，需要知道ip，在知道主机上的进程需要知道端口号，所以要确定网络环境中的一个进程，可以使用ip + port。

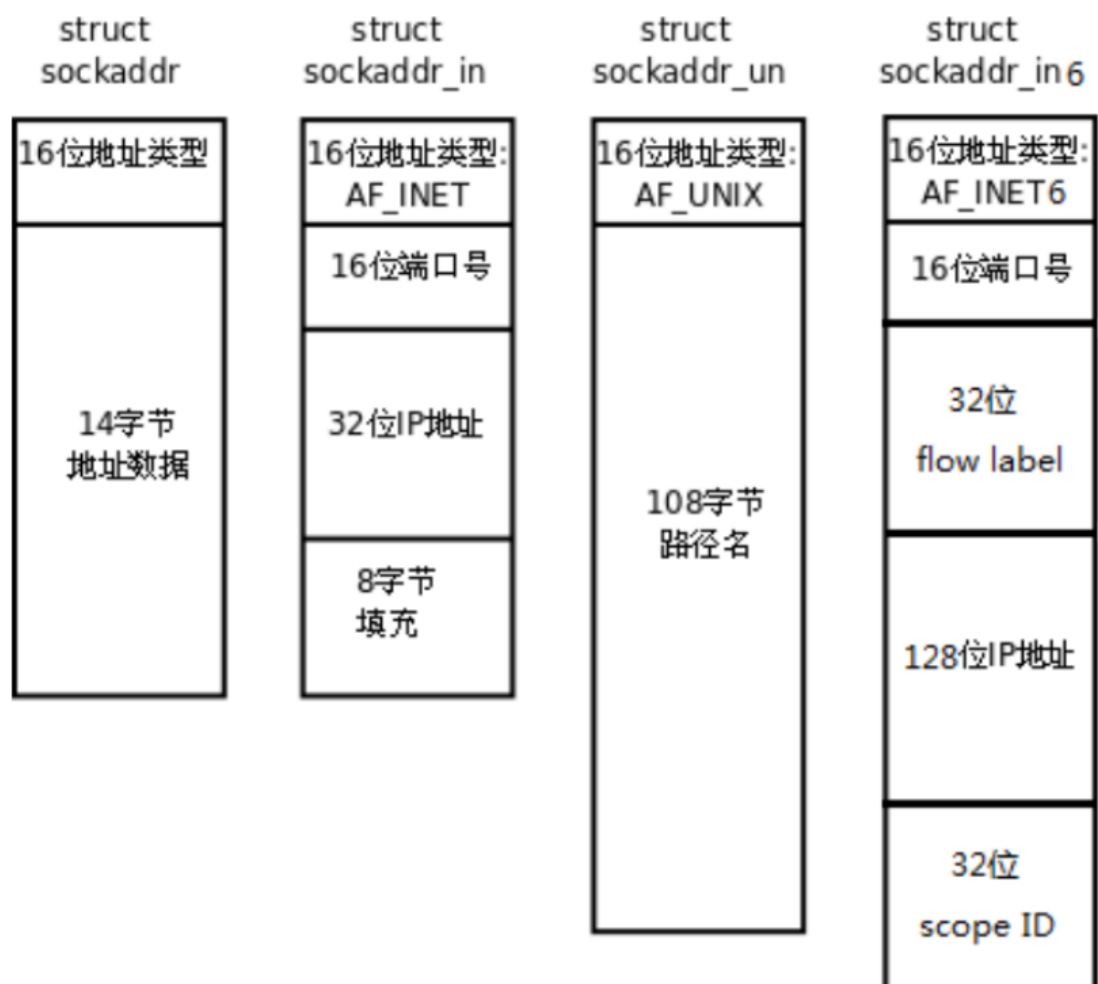
2、字节序

大端：低地址存高位，高地址存低位；小端：低地址存低位，高地址存高位（x86采用小端存储）

TCP/IP协议规定，网络数据流应采用大端字节序

```
1  #include <arpa/inet.h>
2  uint32_t htonl(uint32_t hostlong); //h = host n = network l = long s = short
3  uint16_t htons(uint16_t hostshort);
4  uint32_t ntohl(uint32_t netlong);
5  uint16_t ntohs(uint16_t netshort);
6
7  in_addr_t inet_addr(const char *cp);
8
9  int getsockname(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
10 int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

3、两个重要结构体

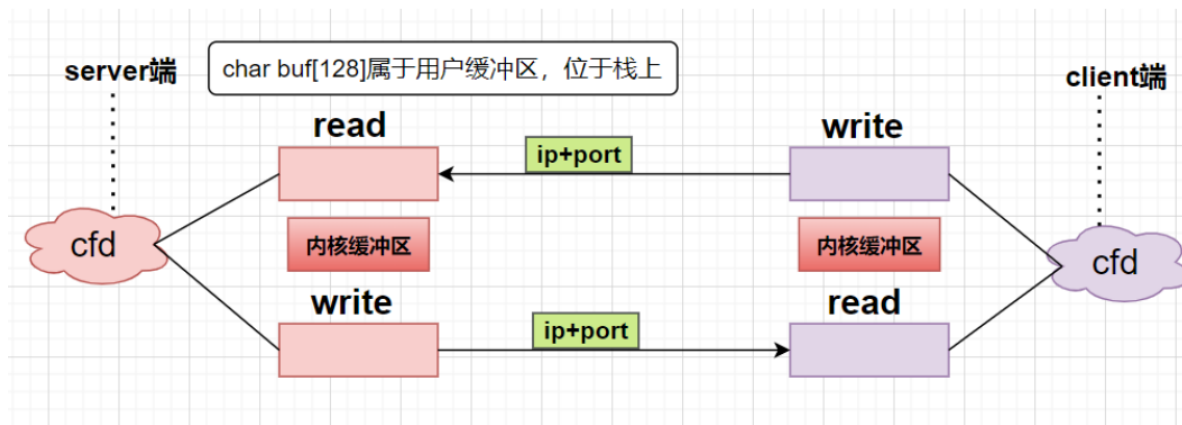


```

1  struct sockaddr
2  {
3      sa_family_t sa_family; /* address family, AF_XXX */
4      char sa_data[14]; /* 14 bytes of protocol address */
5  };
6
7  struct sockaddr_in
8  {
9      sa_family_t sin_family; /* address family: AF_INET */
10     in_port_t sin_port; /* port in network byte order */
11     struct in_addr sin_addr; /* internet address */
12 };
13
14 /* Internet address. */
15 struct in_addr
16 {
17     uint32_t s_addr; /* address in network byte order */
18 };

```

4、网络编程原理图



5、端口复用

```

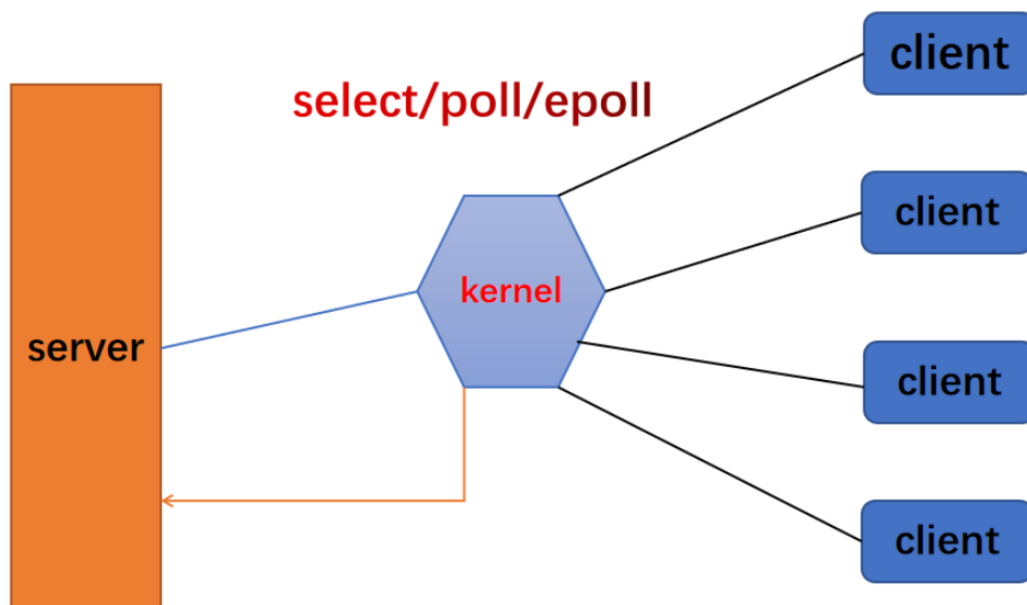
1  int opt = 1;
2  setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
3
4  int opt = 1;
5  setsockopt(listenfd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));

```

六、IO多路复用 (重要)

1、概念图

多进程与多线程并发服务器，不经常使用这种作为大型服务器开发的原因是，所有的监听与访问请求都由服务器操作。可以使用多路IO转接服务器（也叫多任务IO服务器），思想：不再由应用程序自己监视客户端连接，取而代之由内核替应用程序监视文件。



2、select的使用

2.1、函数接口

```

1 #include <sys/select.h>
2 #include <sys/time.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5
6 int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
7           struct timeval *timeout);
8 fd_set:位图。
9
10 void FD_ZERO(fd_set *set); //将set清空为0
11 void FD_SET(int fd, fd_set *set); //将fd设置到set集合中
12 void FD_CLR(int fd, fd_set *set); //将fd从set中清除出去
13 int FD_ISSET(int fd, fd_set *set); //判断fd是否在集合中

```

select使用的原理图

数据结构：位图

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

文件描述符file description,fd 3 5 8

2.2、优缺点

- 1、文件描述符上限（1024），同时监听的文件描述符1024个，历史原因，不好修改，除非重新编译Linux内核。
- 2、当监听的文件描述符个数比较稀疏的时候（比如3，600，1023），循环判断比较麻烦，所以需要自定义数据结构：数组
- 3、监听集合与满足监听条件的集合是同一个，需要将原有集合保存

3、poll的使用

3.1、函数接口

```

1  #include <poll.h>
2  int poll(struct pollfd *fds, nfds_t nfds, int timeout);
3  struct pollfd
4  {
5      int fd; /* file descriptor */
6      short events; /* requested events */
7      short revents; /* returned events */
8  };
9
10 fds: 文件描述符数组。
11 events: POLLIN/POLLOUT/POLLERR
12 nfds: 监控数组中有多少文件描述符需要被监控

```

3.2、优缺点

优点:

- 1、突破文件描述符1024的上限
- 2、监听与返回的集合分离
- 3、搜索范围变小（已经知道是那几个数组）

缺点:

监听1000个文件描述符，但是只有3个满足条件，这样也需要全部遍历，效率依旧低

4、epoll的使用

4.1、函数接口

```

1  #include <sys/epoll.h>
2  int epoll_create(int size);
3
4  int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
5  epfd: epoll_create函数返回的值
6  op: EPOLL_CTL_ADD/EPOLL_CTL_MOD/EPOLL_CTL_DEL
7
8  //等待所监控文件描述符上有事件的产生
9  int epoll_wait(int epfd, struct epoll_event *events, int maxevents,
10                int timeout);
11
12 typedef union epoll_data
13 {
14     void *ptr;
15     int fd;
16     uint32_t u32;
17     uint64_t u64;
18 } epoll_data_t;
19
20 struct epoll_event
21 {
22     uint32_t events; /* Epoll events */
23     epoll_data_t data; /* User data variable */
24 };

```

4.2、优点

- 文件描述符数目没有上限：通过epoll_ctl()来注册一个文件描述符，内核中使用**红黑树**的数据结构来管理所有需要监控的文件描述符。

- 基于事件就绪通知方式：一旦被监听的某个文件描述符就绪，内核会采用类似于**callback的回调机制**，迅速激活这个文件描述符，这样随着文件描述符数量的增加，也不会影响判定就绪的性能。
- 维护就绪队列：当文件描述符就绪，就会被放到内核中的一个**就绪队列**中，这样调用epoll_wait获取就绪文件描述符的时候，只要取队列中的元素即可，操作的时间复杂度恒为 $O(1)$

Day06

一、问题回顾

- 1、C++11线程库中线程的入口函数的传递方式有哪些？
- 2、C++11线程库中互斥锁是哪个类？lock_guard与unique_lock使用了什么思想？特点是什么？
- 3、C++11线程库中条件变量与那个锁类型一起使用？
- 4、OSI有哪七层？TCP协议的特点？什么是三次握手与四次挥手？
- 5、状态迁移图有哪11种状态？
- 6、字节序有那两种方式？
- 7、有哪三种IO多路复用？各有哪些函数？

二、五种网络IO模型（重要）

1、阻塞式IO

阻塞式I/O模型

在两个阶段，应用进程都会处于阻塞状态

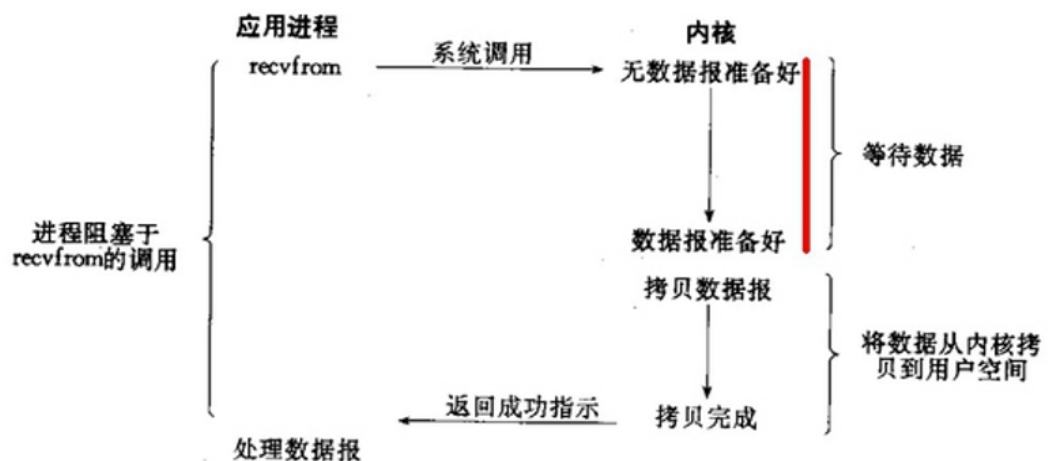


图 6.1 阻塞 I/O 模型

2、非阻塞IO

非阻塞式I/O

应用进程在第一阶段“等待数据阶段”不会阻塞，会轮询查看，但是在第二阶段“数据从内核态拷贝到用户态”会阻塞



图 6.2 非阻塞 I/O 模型

3、IO多路复用

I/O多路复用

在第一阶段，应用进程会阻塞等待，但是可以同时监听多个文件描述符。当IO多路复用有正确返回值的时候，在第二个阶段，相当于有多个文件描述符同时在操作。

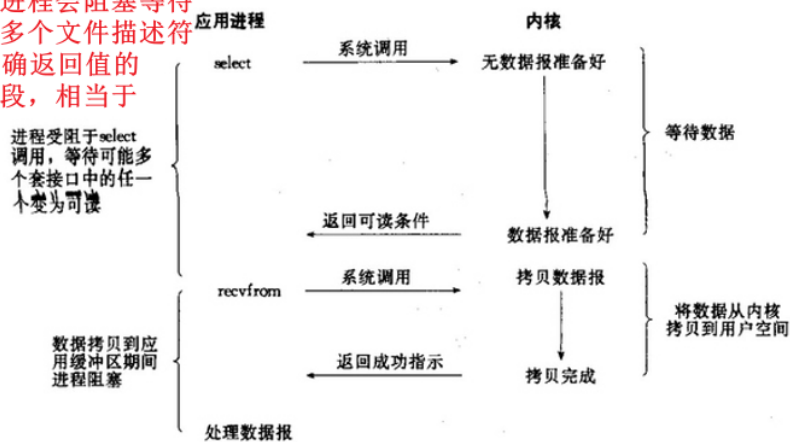


图 6.3 I/O 复用模型

4、信号驱动IO

信号驱动式I/O

应用进程在第一阶段注册了信号之后，会立马返回，当内核准备好数据后会通知应用进程，然后进程在第二阶段会阻塞等待数据从内核态拷贝到用户态



图 6.4 信号驱动 I/O 模型

总结：在这四种IO模型中，应用进程在第二阶段都是出于阻塞状态的。

5、异步IO

异步I/O

在两个阶段，应用进程都处于非阻塞状态

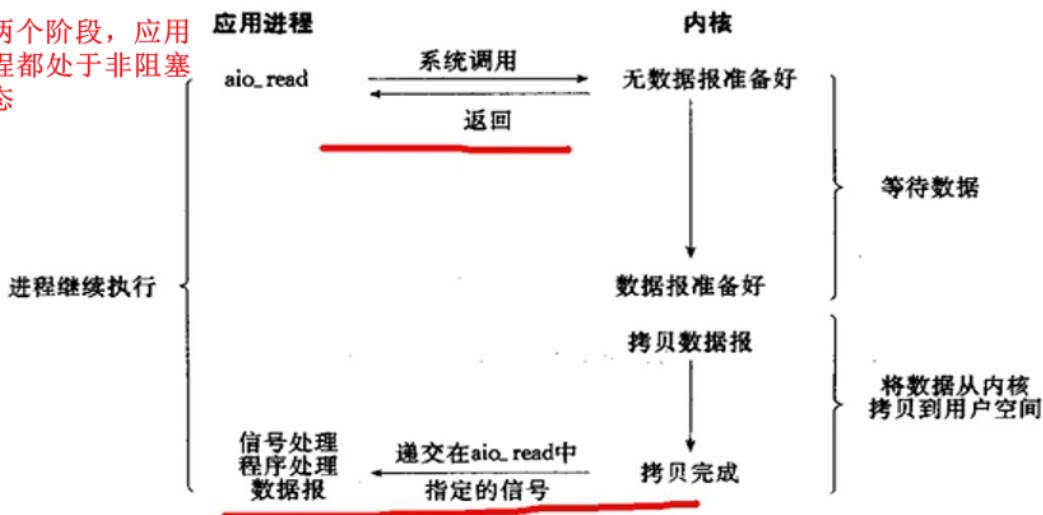
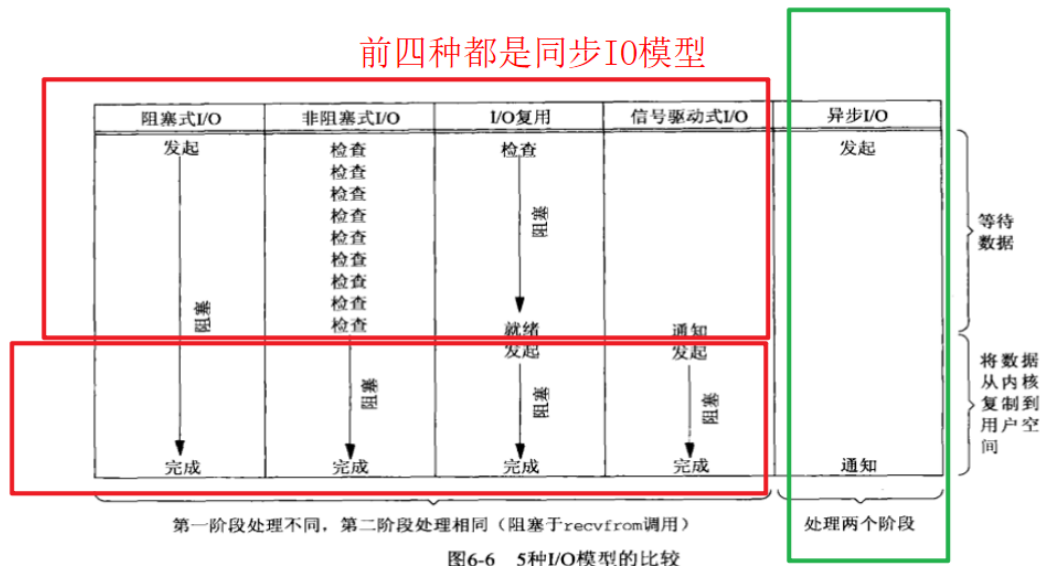


图 6.5 异步 I/O 模型

比较结果



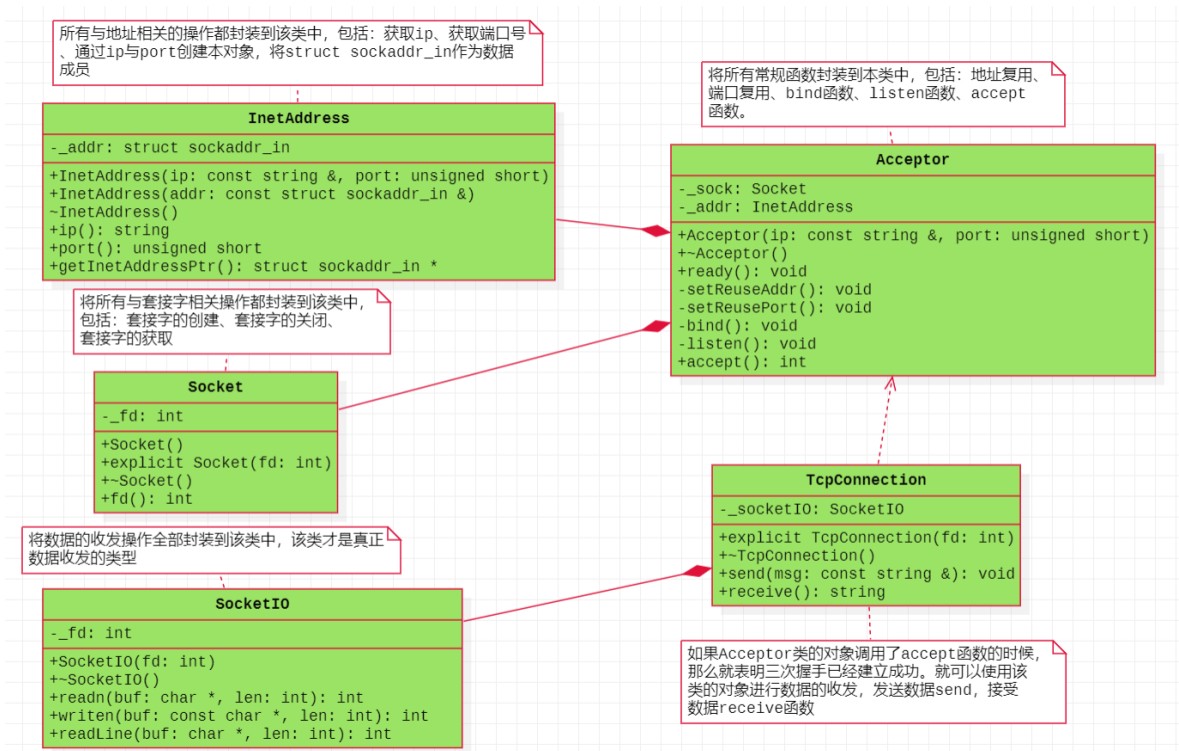
三、ReactorV1版本

- 1、Socket类：将所有与套接字相关操作都封装到该类中，包括：套接字的创建、套接字的关闭、套接字的获取。
- 2、InetAddress类：所有与地址相关的操作都封装到该类中，包括：获取ip、获取端口号、通过ip与port创建本对象，将struct sockaddr_in作为数据成员。
- 3、Acceptor类：将所有常规函数封装到本类中，包括：地址复用、端口复用、bind函数、listen函数、accept函数。
- 4、TcpConnection类：如果Acceptor类的对象调用了accept函数的时候，那么就表明三次握手已经建立成功。就可以使用该类的对象进行数据的收发，发送数据send，接受数据receive函数。
- 5、SocketIO类：将数据的收发操作全部封装到该类中，该类才是真正数据收发的类型。

```
1  recv与read的区别
2  ssize_t recv(int sockfd, void *buf, size_t len, int flags);
3  recv与read的区别在与flags参数，如果flags为0的话，那么recv与read是完全等价的。
4  如果flags = MSG_PEEK，就会将数据从缓冲区读出来，但是不会将数据从缓冲区中移除。
```

两个函数

```
1  #include <sys/socket.h>
2  //可以通过文件描述符fd获取本端的地址信息，该信息会存放在第二个参数addr中
3  int getsockname(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
4
5  //可以通过文件描述符fd获取对端的地址信息，该信息会存放在第二个参数addr中
6  int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```



四、ReactorV2版本

1、伪代码

```

1 void EventLoop::loop()
2 {
3     _isLooping = true;
4     while(_isLooping)
5     {
6         waitEpollFd();
7     }
8 }
9
10 void EventLoop::unloop()
11 {
12     _isLooping = false;
13 }
14
15 void EventLoop::waitEpollFd()
16 {
17     nready = epoll_wait();
18     if(-1 == nready && errno == EINT)
19     {
20     }
21     else if(-1 == nready)
22     {
23     }
24     else if(0 == nready)
25     {
26     }
27     else
28     {
29     }
30 }
31
  
```

```

32         for(size_t idx = 0; idx < nready; ++idx)
33         {
34             if(文件描述符 == listenfd)
35             {
36                 //处理新的连接
37                 handleNewConnection();
38             }
39             else
40             {
41                 //处理老的连接
42                 handleMessage();
43             }
44         }
45     }
46 }
47
48 void EventLoop::handleNewConnection()
49 {
50     //三次握手建立成功
51     fd = _acceptor.accept();
52     //使用文件描述符创建连接con
53     TcpConnection con(fd);
54
55     //将键值对存放在map
56     _conns.insert(fd, con);
57 }
58
59 void EventLoop::handleMessage(int fd)
60 {
61     it = _conns.find(fd);
62     if(it != _conns.end())
63     {
64         //连接是存在的，可以进行数据的收发
65     }
66     else
67     {
68         //连接不存在，可以直接让程序退出来
69     }
70 }

```

Day07

一、ReactorV2版本

1、三个半事件

TCP网络编程最本质的是处理三个半事件：

连接建立：包括服务器端被动接受连接（accept）和客户端主动发起连接（connect）。TCP连接一旦建立，客户端和服务端就是平等的，可以各自收发数据。

连接断开：包括主动断开（close、shutdown）和被动断开（read()返回0）。

消息到达：文件描述符可读。这是最为重要的一个事件，对它的处理方式决定了网络编程的风格（阻塞还是非阻塞，如何处理分包，应用层的缓冲如何设计等等）。

消息发送完毕：这算半个。对于低流量的服务，可不必关心这个事件；另外，这里的"发送完毕"是指数据写入操作系统缓冲区（内核缓冲区），将由TCP协议栈负责数据的发送与重传，不代表对方已经接收到数据。

2、类图



该类的作用是为了封装epoll，而epoll有三个函数，所以在该类中至少需要将这三个函数封装进来。epoll_ctl与epoll_wait会用到epoll_create的返回结果，所以将该返回结果作为数据成员_epfd，对于epoll_wait而言，会将满足条件的文件描述符放在一个结构体数组中，所以使用vector将其存起来，也就有了_evtList。整个epoll的核心代码都在while循环中，为何标识循环是否在运行，可以设置标志位_isLooping。如果监听的文件描述符是listenfd的时候，就表明有新的连接进来了，就需要接受新的连接，也就是需要调用accept函数，但是该函数已经封装到Acceptor类中了，所以需要Acceptor的对象。如果accept函数有正确的返回结果，就表明三次握手建立成功，就表明可以创建一条连接，为了更好的管理连接，使用了智能指针进行管理，最终就有了map<int, shared_ptr<TcpConnection>>



如果Acceptor类的对象调用了accept函数的时候，那么就表明三次握手已经建立成功。就可以使用该类的对象进行数据的收发，发送数据send，接受数据receive函数

3、添加三个半事件（回调）

Tcp网络编程中虽然有三个半事件，但是我们只关注其中的前三个，即：连接的建立、消息到达、连接断开。三个事件都是与连接相关的，也就是与TcpConnection相关，这三个事件可以使用回调机制的形式进行，也就是回调函数的注册与回调函数的执行，但是现在TcpConnection对象的创建在EventLoop中，所以三个事件需要先传递给EventLoop，再交给TcpConnection，而在C++中，回调函数可以使用bind与function结合使用，也就是function可以绑定函数类型，而现在每个事件都与连接相关，所以就是function<void(const TcpConnectionPtr &)>,也就是可以将其改名，using TcpConnectionCallback = function<void(const TcpConnectionPtr &)>.然后就是EventLoop注册三个回调函数，并且作为桥梁传递给TcpConnection对象，但是EventLoop不会执行该回调函数，因为三个事件的执行只与TcpConnection连接相关，与EventLoop没有关系。最后的结果是，在TcpConnection与EventLoop都注册三个回调函数与设置三个数据成员，但是只有TcpConnection中有三个回调函数的执行。

4、重要代码解析

4.1、数据成员键值对存储

```
17 using TcpConnectionPtr = shared_ptr<TcpConnection>;
18 using TcpConnectionCallback = function<void(const TcpConnectionPtr &)>;
19
20 class EventLoop
```

```

55     int _epfd;//epoll_create创建的文件描述符
56     vector<struct epoll_event> _evtList;//存放满足条件的文件描述符的数据结构
57     bool _isLooping;//标识循环是否在运行的标志
58     Acceptor & acceptor;//因为需要调用其中的accept函数
59     map<int, TcpConnectionPtr> _conns;//存放的是文件描述符与连接的键值对
60

```

4.2、三个回调的存储

需要在EventLoop中与TcpConnection分别存储

```

60     EventLoop类中，添加三个数据成员
61     TcpConnectionCallback _onNewConnectionCb;//连接建立
62     TcpConnectionCallback _onMessageCb;//消息到达
63     TcpConnectionCallback _onCloseCb;//连接断开
64
46 private:
47     SocketIO _sockIO;
48
49     //为了调试而加入的三个数据成员
50     Socket _sock;
51     InetAddress _localAddr;
52     InetAddress _peerAddr;
53     TcpConnection类中添加三个数据成员
54     TcpConnectionCallback _onNewConnectionCb;//连接建立
55     TcpConnectionCallback _onMessageCb;//消息到达
56     TcpConnectionCallback _onCloseCb;//连接断开

```

4.3、三个事件在EventLoop注册

```

195 void EventLoop::setNewConnectionCallback(TcpConnectionCallback &&cb)
196 {
197     _onNewConnectionCb = std::move(cb);
198 }
199 //三个事件在EventLoop中注册
200 void EventLoop::setMessageCallback(TcpConnectionCallback &&cb)
201 {
202     _onMessageCb = std::move(cb);
203 }
204
205 void EventLoop::setCloseCallback(TcpConnectionCallback &&cb)
206 {
207     _onCloseCb = std::move(cb);
208 }

```

4.4、三个事件在TcpConnection注册与执行

在TcpConnection中注册的时候，需要使用左值，不然后面的连接进来之后，就得不到回调函数的值了。

```

31     //三个回调的注册
32     //三个事件在TcpConnection中注册与执行
33     void setNewConnectionCallback(const TcpConnectionCallback &cb);
34     void setMessageCallback(const TcpConnectionCallback &cb);
35     void setCloseCallback(const TcpConnectionCallback &cb);
36
37     //三个回调的执行
38     void handleNewConnectionCallback();
39     void handleMessageCallback();
40     void handleCloseCallback();

```

4.5、防止智能指针的误用

```

18 class TcpConnection
19 : public std::enable_shared_from_this<TcpConnection>
20 {
21 public:
102 void TcpConnection::handleNewConnectionCallback()
103 {
104     if(_onNewConnectionCb)
105     {
106         /* onNewConnectionCb(shared_ptr<TcpConnection>(this)); */
107         _onNewConnectionCb(shared_from_this());
108     }
109     else
110     {
111         cout << "_onNewConnectionCb == nullptr" << endl;
112     }
113 }

```

4.6、EventLoop中epoll_wait参数问题

```

44 //封装类epoll_wait函数
45 void EventLoop::waitEpollFd()
46 {
47     int nready = 0;
48     do
49     {
50         nready = epoll_wait(_epfd, &*_evtList.begin(), _evtList.size(), 3000);
51     }while(-1 == nready && errno == EINTR);
52
53     if(-1 == nready)
54     {
55         cout << "Error: epoll_wait failed" << endl;

```

4.7、EventLoop处理新的连接请求

```

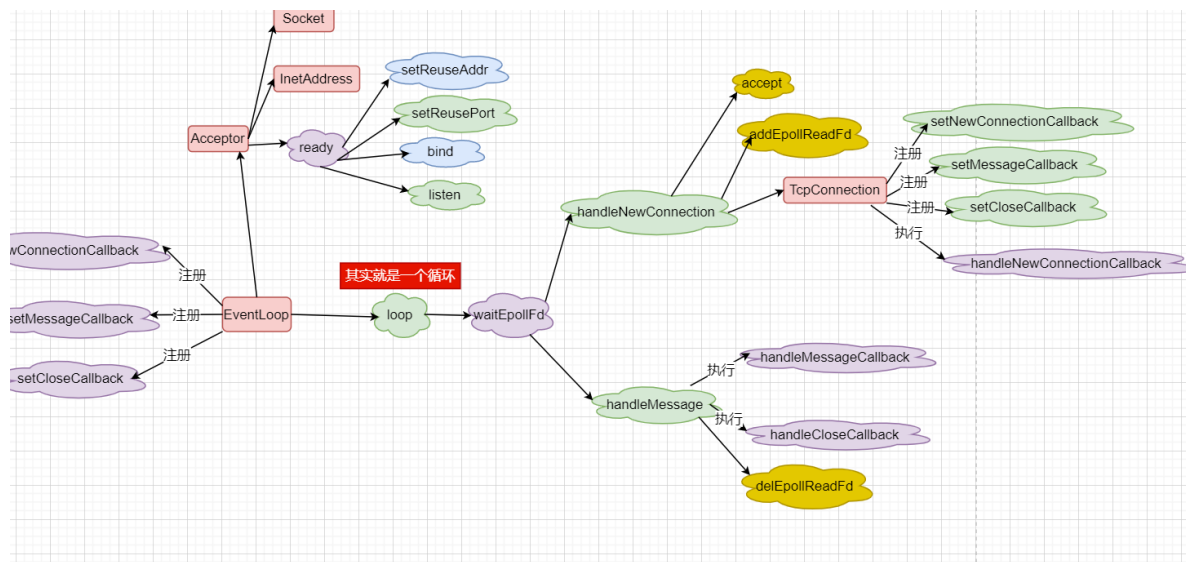
95 //处理新的连接
96 void EventLoop::handleNewConnection()
97 {
98     int connfd = _acceptor.accept(); //阻塞等待客户端的连接，只要该函数有正确
99     if(connfd < 0) //返回结果，就表明三次握手已经完全完成了
100     { //就可以建立连接了
101         perror("handleNewConnection accept");
102         return;
103     }
104
105     addEpollReadFd(connfd); //将connfd放在红黑树上进行监听
106
107     //就表明三次握手已经建立成功了
108     /* TcpConnection con(connfd); */ //使用该连接创建TcpConnection对象
109     TcpConnectionPtr con(new TcpConnection(connfd)); //注册回调函数
110
111     //将三个回调函数注册给TcpConnection
112     con->setNewConnectionCallback(_onNewConnectionCb); //连接建立的注册
113     con->setMessageCallback(_onMessageCb); //消息到达的注册
114     con->setCloseCallback(_onCloseCb); //连接断开的注册
115
116     //以键值对的形式存起来
117     /* _conns.insert(std::make_pair(connfd, con)); */
118     _conns[connfd] = con; //将连接对应的文件描述符与连接放在容器中存起来
119
120     con->handleNewConnectionCallback(); //连接建立的事件触发的时机已经到达
121 }

```

4.8、EventLoop中老的连接上有数据收发

```
123 //处理老的连接上的消息
124 void EventLoop::handleMessage(int fd)
125 {
126     auto it = _conns.find(fd); 使用fd在容器中找到TcpConnection连接
127     if(it != _conns.end())
128     {
129         //连接是存在的，可以进行数据的收发
130         //read/recv = ret = 0 使用该函数查看是不是断开
131         bool flag = it->second->isClosed(); //读的时候客户端是不是与服务器断开
132         if(flag)
133         {
134             //连接已经断开 断开就触发连接断开的事件，并将其从红黑树上删除监听，并从容器中删除掉
135             it->second->handleCloseCallback(); //连接断开的事件的触发时机已经到达
136             delEpollReadFd(fd); //将文件描述符从红黑树上摘除掉
137             _conns.erase(it); //同时将该链接从map中删除
138         }
139         else
140         {
141             it->second->handleMessageCallback(); //消息到达事件的触发时机已经到达
142         }
143     }
144 }
```

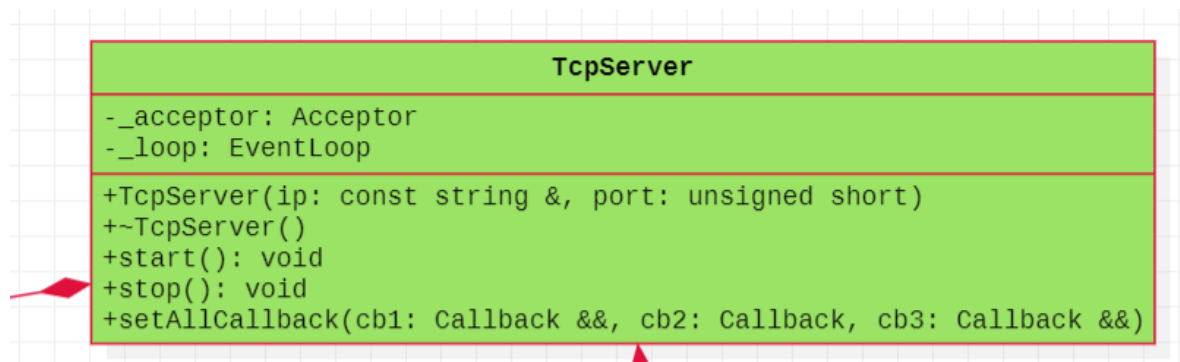
5、流程图



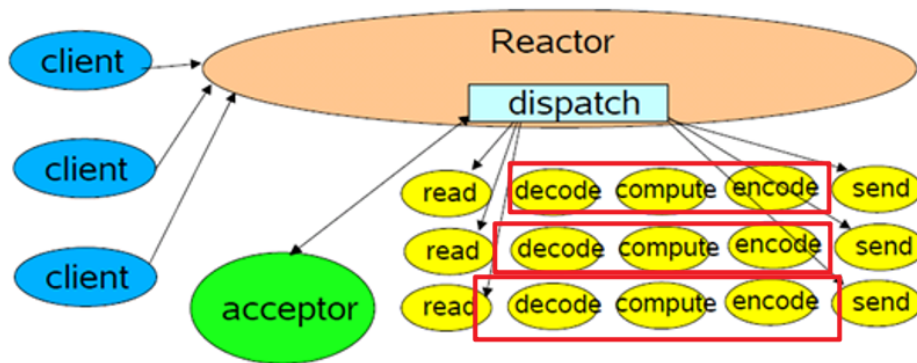
Day08

一、ReactorV3版本

1、类图



Basic Reactor Design



2、ReactorV3版本的缺陷

- 如果业务逻辑的处理比较复杂，那么V3版本中的onMessage函数中就不能处理业务逻辑。
- 可以将业务逻辑的处理交给线程池去做。需要将msg交给线程池，但是如果msg处理好之后，也要发送给EventLoop，而发送数据的能力只有TcpConnection，所以就将连接与消息都发送给线程池。
- 就可以将连接TcpConnection的对象con与需要发送的数据msg打包交给MyTask，然后在MyTask的process函数处理msg这个业务逻辑。然后业务逻辑处理好之后通过con进行发送给EventLoop
- 那么线程池在处理好数据之后，就可以立马进行发送给EventLoop，所以需要通知EventLoop接收处理好之后的数据，也就是需要让线程池与EventLoop之间碱性通信。
- 进程或者线程之间进行通信的方式有一种：eventfd

二、eventfd的使用

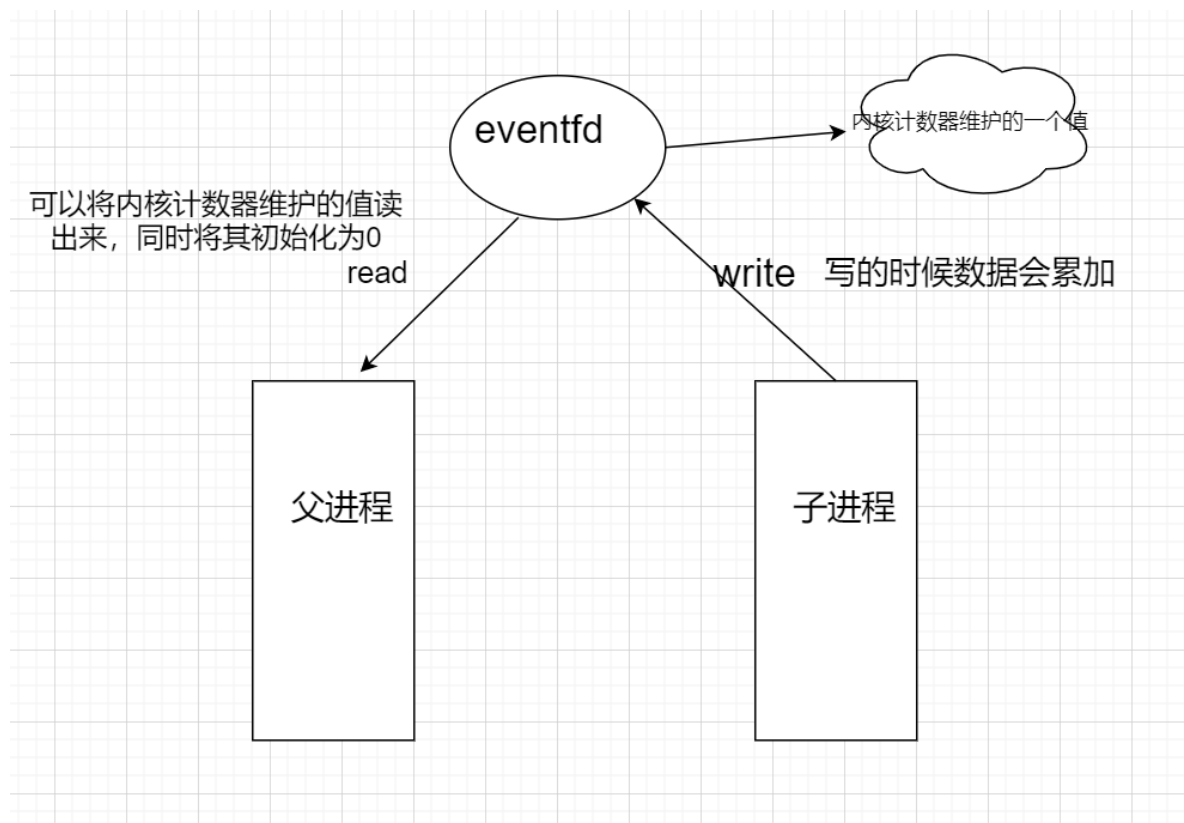
1、作用

主要用于进程或者线程之间进行通信

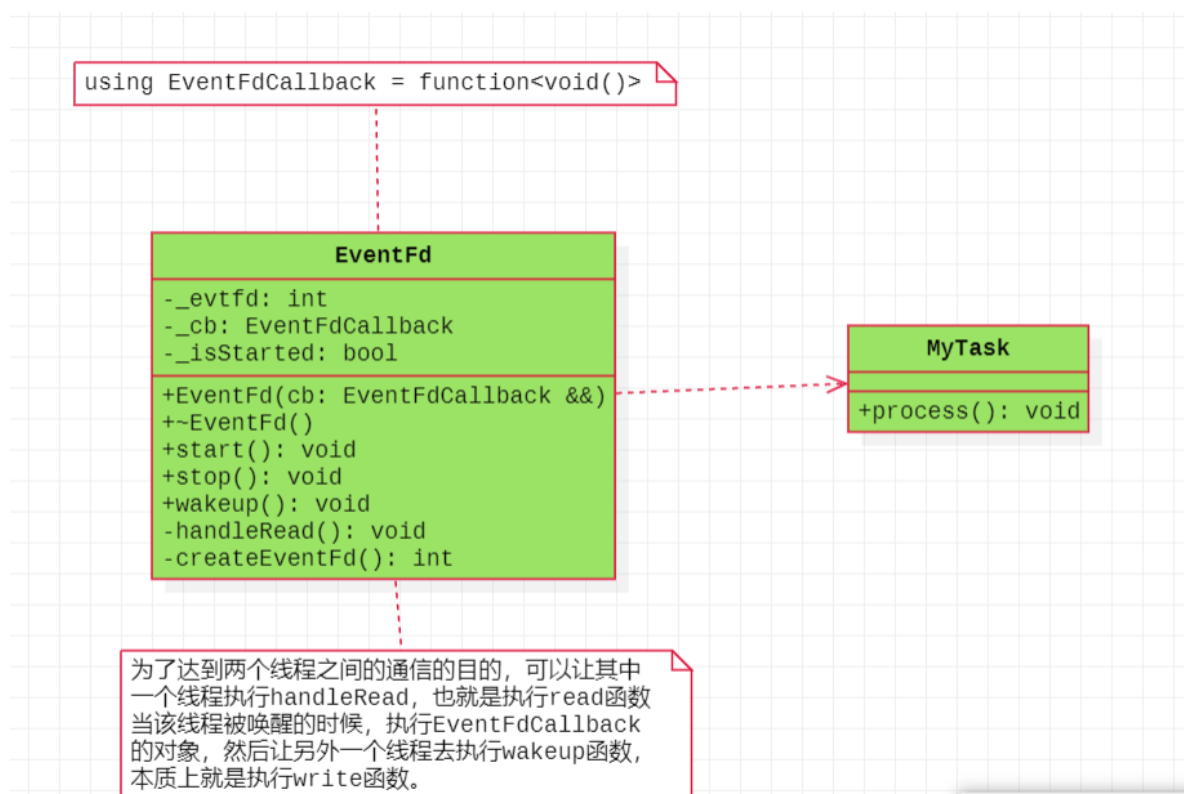
2、函数接口

```
1 #include <sys/eventfd.h>
2 int eventfd(unsigned int initval, int flags);
3 initval: 内核计数器的初始值。
4 flags: 可以设置为0
5
6 返回值: 函数返回一个文件描述符，与打开的其他文件一样，可以进行读写操作
```

3、进程间通信的原理图



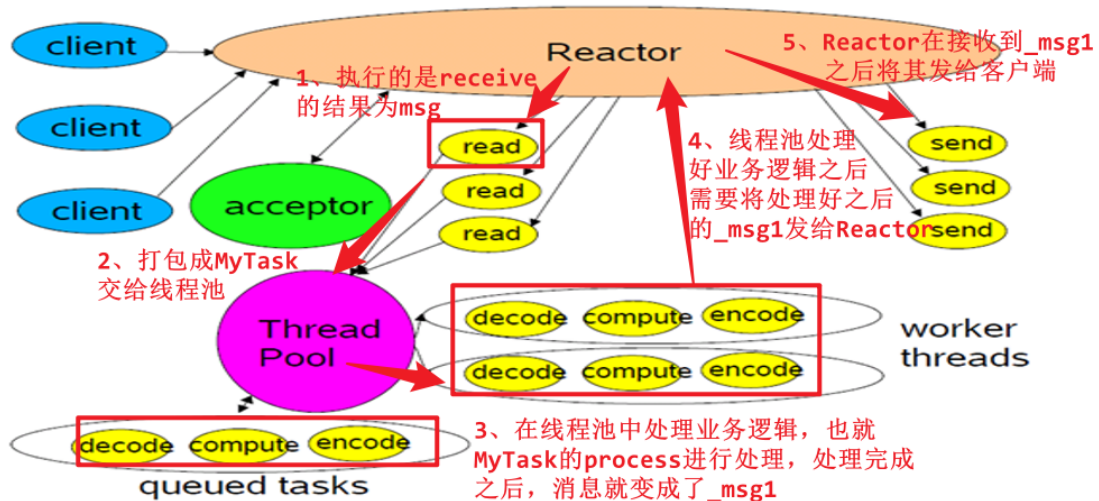
4、eventfd的封装



三、ReactorV4版本

1、原理图

Worker Thread Pools



2、类图



```

TcpConnection
-__loop: EventLoop *
-__socketIO: SocketIO
-__sock: Socket
-__localAddr: InetAddress
-__peerAddr: InetAddress
-__onNewConnectionCb: TcpConnectionCallback
-__onMessageCb: TcpConnectionCallback
-__onCloseCb: TcpConnectionCallback

+explicit TcpConnection(fd: int, loop: EventLopp *)
+~TcpConnection()
+send(msg: const string &): void
+receive(): string
+toString(): string
+isClosed(): bool
+getLocalAddr(): InetAddress
+getPeerAddr(): InetAddress
+setNewConnectionCallback(cb: const TcpConnectionCallback &): void
+setMessageCallback(cb: const TcpConnectionCallback &): void
+setCloseCallback(cb: const TcpConnectionCallback &): void
+handleNewConnectionCallback(): void
+handleMessageCallback(): void
+handleCloseCallback(): void
+sendInLoop(msg: const string &): void

```

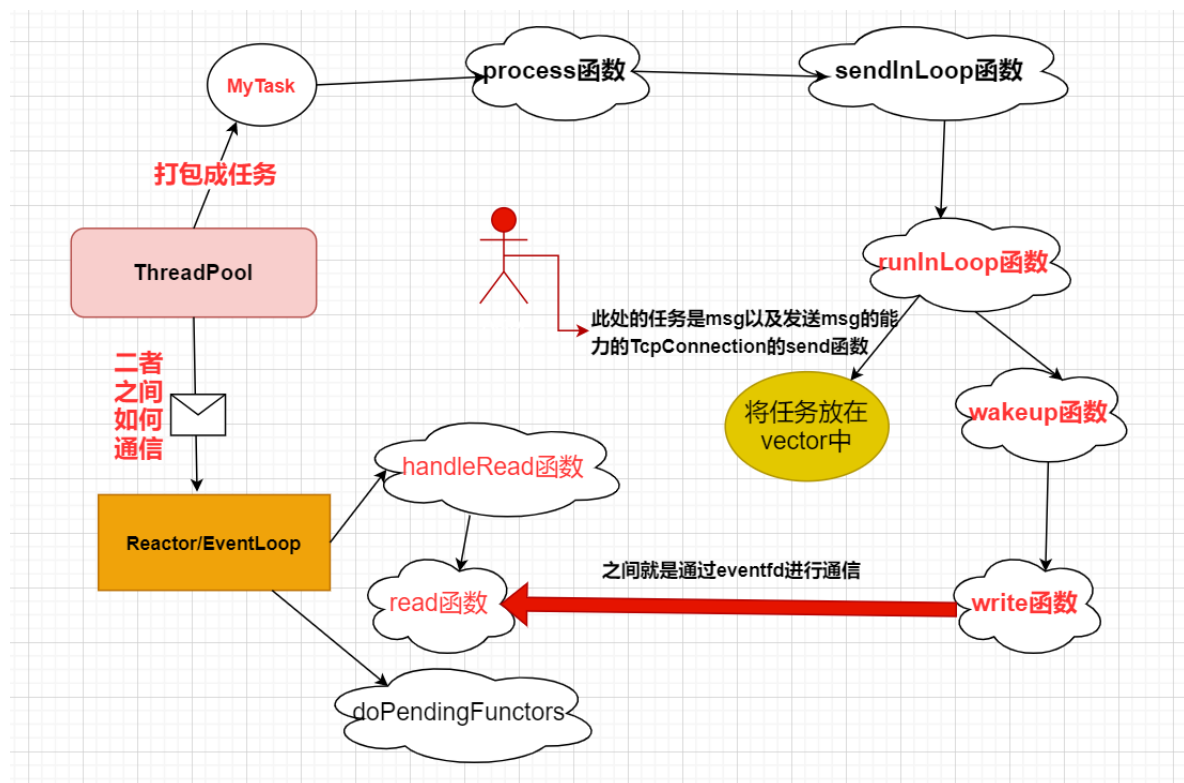
Tcpconnection需要知道**EventLoop**的存在，因为需要将处理好之后的数据发给**EventLoop**

增加参数，初始化数据成员**_loop**

目的就是将发送数据能力的**send**以及处理好的数据**msg**发给**EventLoop**

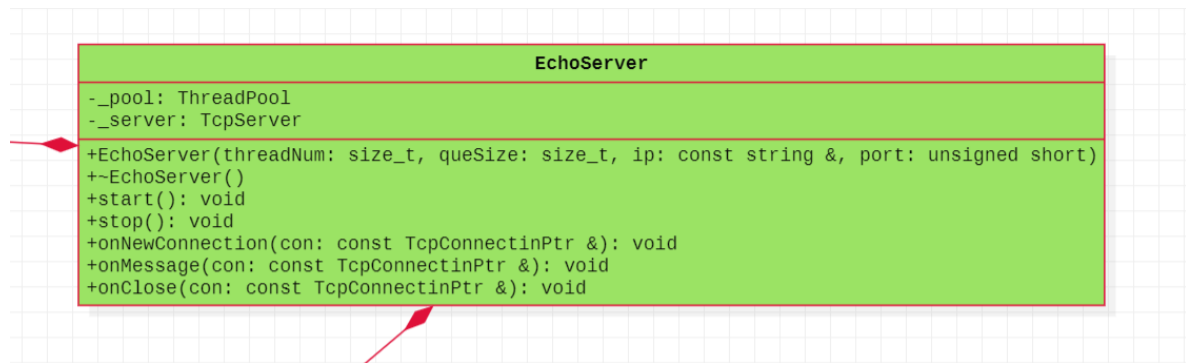
如果Acceptor类的对象调用了accept函数的时候，那么就表明三次握手已经建立成功。就可以使用该类的对象进行数据的收发，发送数据send，接受数据receive函数

3、流程图



一、ReactorV5版本

1、类图



```
33 void EchoServer::start()
34 {
35     _pool.start();
36     using namespace std::placeholders;
37     _server.setAllCallback(std::bind(&EchoServer::onNewConnection, this, _1)
38                             , std::bind(&EchoServer::onMessage, this, _1)
39                             , std::bind(&EchoServer::onClose, this, _1));
40     _server.start();
41 }
42
```

总结:

- 1、read与send是从网络读数据或者将数据发送到网络上的，也就是进行网络的IO操作，将处理网络IO操作的线程称为IO线程。
- 2、业务逻辑的处理decode、compute、encode的线程称为计算线程，需要有CPU的参数。
- 3、如果计算花费的时间非常的少的话，此时可以使用基本的reactor模型。
- 4、如果计算花费的事件非常长的话，也就是计算线程耗时比较严重，那么建议使用Reactor + ThreadPool.

二、timerfd的使用

1、作用

这个接口基于文件描述符，通过文件描述符的可读事件进行超时通知，所以能够被用于select/poll/epoll的应用场景

2、函数接口

```
1 #include <sys/timerfd.h>
2 int timerfd_create(int clockid, int flags);
3 功能：该函数生成一个定时器对象，返回与之关联的文件描述符。
4 参数详解：
5 clockid: 可设置为
6 CLOCK_REALTIME: 相对时间，从1970.1.1到目前的时间。更改系统时间 会更改获取的值，它以系统时间为坐标。
7 CLOCK_MONOTONIC: 绝对时间，获取的时间为系统重启到现在的时间，更改系统时间对齐没有影响。
8 flags: 可设置为
9 TFD_NONBLOCK (非阻塞)，
10 TFD_CLOEXEC (同O_CLOEXEC)
```

```

11  Linux内核2.6.26版本以上都指定为0
12
13
14  int timerfd_settime(int fd, int flags, const struct itimerspec *new_value,
15                      struct itimerspec *old_value);
16  功能: 该函数能够启动和停止定时器
17  参数详解:
18  fd: timerfd对应的文件描述符
19  flags:
20  0表示是相对定时器
21  TFD_TIMER_ABSTIME表示是绝对定时器
22  new_value: 设置超时时间, 如果为0则表示停止定时器。
23  old_value:
24  一般设为NULL, 不为NULL,
25  则返回定时器这次设置之前的超时时间
26
27  struct timespec
28  {
29      time_t tv_sec;           /* Seconds */
30      long tv_nsec;           /* Nanoseconds */
31  };
32
33  struct itimerspec
34  {
35      struct timespec it_interval; /* Interval for periodic timer */
36      struct timespec it_value;    /* Initial expiration */
37  };

```

3、类图

